Chương 11 – Huấn luyện Mạng thần kinh sâu (Deep Neural Networks)

Notebook này chứa tất cả mã mẫu và lời giải cho các bài tập trong chương 11.



Thiết lập

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Và TensorFlow ≥ 2.8:

```
from packaging import version
import tensorflow as tf

assert version.parse(tf.__version__) >= version.parse("2.8.0")
```

Tương tự như các chương trước, hãy định nghĩa kích thước phông chữ mặc định để làm cho các hình vẽ đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsize=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)
```

Và hãy tạo thư mục `images/deep` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` được sử dụng xuyên suốt notebook này để lưu các hình vẽ với độ phân giải cao cho sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "deep"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

Vấn đề Triệt tiêu/Bùng nổ Độ dốc

```
# extra code – this cell generates and saves Figure 11–1

import numpy as np

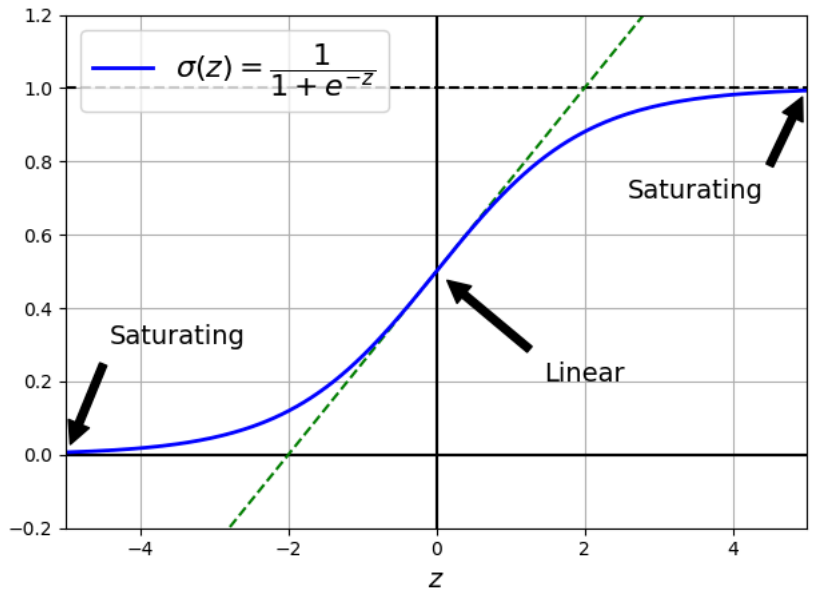
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-5, 5, 200)

plt.plot([-5, 5], [0, 0], 'k-')
plt.plot([-5, 5], [1, 1], 'k--')
plt.plot([0, 0], [-0.2, 1.2], 'k-')
plt.plot([-5, 5], [-3/4, 7/4], 'g--')
plt.plot(z, sigmoid(z), "b-", linewidth=2,
         label=r"$\sigma(z) = \dfrac{1}{1+e^{-z}}$")
```

```
props = dict(facecolor='black', shrink=0.1)
plt.annotate('Saturating', xytext=(3.5, 0.7), xy=(5, 1), arrowprops=props,
             fontsize=14, ha="center")
plt.annotate('Saturating', xytext=(-3.5, 0.3), xy=(-5, 0), arrowprops=props,
             fontsize=14, ha="center")
plt.annotate('Linear', xytext=(2, 0.2), xy=(0, 0.5), arrowprops=props,
             fontsize=14, ha="center")
plt.grid(True)
plt.axis([-5, 5, -0.2, 1.2])
plt.xlabel("$z$")
plt.legend(loc="upper left", fontsize=16)

save_fig("sigmoid_saturation_plot")
plt.show()
```



Khởi tạo Xavier và He

```
dense = tf.keras.layers.Dense(50, activation="relu",
                              kernel_initializer="he_normal")
```

```
he_avg_init = tf.keras.initializers.VarianceScaling(scale=2., mode="fan_avg",
                                                    distribution="uniform")
dense = tf.keras.layers.Dense(50, activation="sigmoid",
                              kernel_initializer=he_avg_init)
```

Hàm Kích hoạt Không Bão hòa

Leaky ReLU

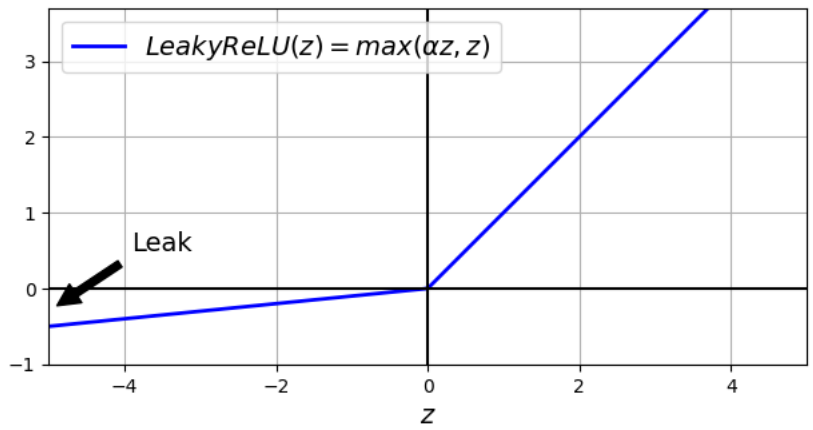
```
# extra code – this cell generates and saves Figure 11–2

def leaky_relu(z, alpha):
    return np.maximum(alpha * z, z)

z = np.linspace(-5, 5, 200)
plt.plot(z, leaky_relu(z, 0.1), "b-", linewidth=2, label=r"$LeakyReLU(z) = max(\alpha z, z)$")
plt.plot([-5, 5], [0, 0], 'k-')
plt.plot([0, 0], [-1, 3.7], 'k-')
plt.grid(True)
props = dict(facecolor='black', shrink=0.1)
plt.annotate('Leak', xytext=(-3.5, 0.5), xy=(-5, -0.3), arrowprops=props,
             fontsize=14, ha="center")
plt.xlabel("$z$")
plt.axis([-5, 5, -1, 3.7])
plt.gca().set_aspect("equal")
```

```
plt.legend()

save_fig("leaky_relu_plot")
plt.show()
```




```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2)  # defaults to alpha=0.3
dense = tf.keras.layers.Dense(50, activation=leaky_relu,
                              kernel_initializer="he_normal")
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alpha` is deprecat
  warnings.warn(
```

```
model = tf.keras.models.Sequential([
    # [...]  # more layers
    tf.keras.layers.Dense(50, kernel_initializer="he_normal"),  # no activation
    tf.keras.layers.LeakyReLU(alpha=0.2),  # activation as a separate layer
    # [...]  # more layers
])
```

ELU

Việc triển khai ELU trong TensorFlow là rất đơn giản, chỉ cần chỉ định hàm kích hoạt khi xây dựng mỗi lớp, và sử dụng khởi tạo He:

```
dense = tf.keras.layers.Dense(50, activation="elu",
                              kernel_initializer="he_normal")
```

SELU

Theo mặc định, các siêu tham số của SELU (`scale` và `alpha`) được điều chỉnh theo cách sao cho giá trị trung bình đầu ra của mỗi nơ-ron vẫn gần bằng 0, và độ lệch chuẩn vẫn gần bằng 1 (giả sử các đầu vào cũng được chuẩn hóa với giá trị trung bình 0 và độ lệch chuẩn 1, và các ràng buộc khác được tôn trọng, như đã giải thích trong sách). Sử dụng hàm kích hoạt này, ngay cả mạng nơ-ron sâu 1.000 lớp cũng giữ được xấp xỉ giá trị trung bình 0 và độ lệch chuẩn 1 qua tất cả các lớp, tránh được vấn đề bùng nổ/triệt tiêu độ dốc:

```
# extra code – this cell generates and saves Figure 11–3

from scipy.special import erfc

# alpha and scale to self normalize with mean 0 and standard deviation 1
# (see equation 14 in the paper):
alpha_0_1 = -np.sqrt(2 / np.pi) / (erfc(1 / np.sqrt(2)) * np.exp(1 / 2) - 1)
scale_0_1 = (
    (1 - erfc(1 / np.sqrt(2)) * np.sqrt(np.e))
    * np.sqrt(2 * np.pi)
    * (
        2 * erfc(np.sqrt(2)) * np.e ** 2
        + np.pi * erfc(1 / np.sqrt(2)) ** 2 * np.e
        - 2 * (2 + np.pi) * erfc(1 / np.sqrt(2)) * np.sqrt(np.e)
        + np.pi
        + 2
    ) ** (-1 / 2)
```

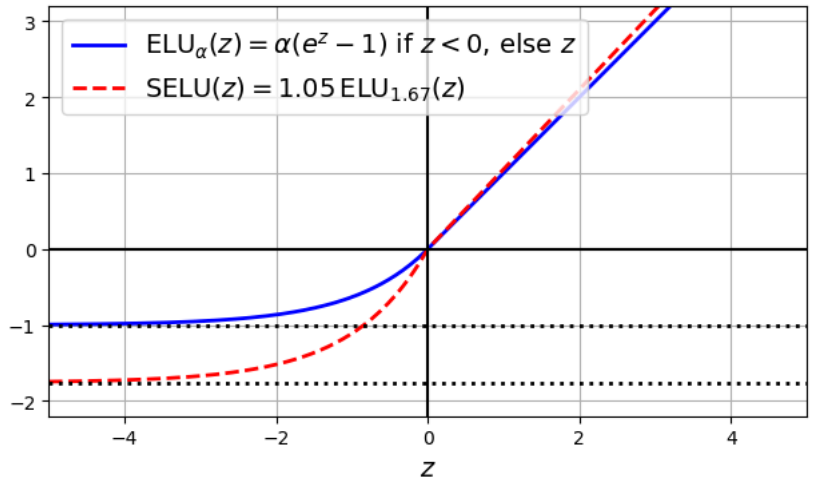
```
)

def elu(z, alpha=1):
    return np.where(z < 0, alpha * (np.exp(z) - 1), z)

def selu(z, scale=scale_0_1, alpha=alpha_0_1):
    return scale * elu(z, alpha)

z = np.linspace(-5, 5, 200)
plt.plot(z, elu(z), "b-", linewidth=2, label=r"ELU$_\alpha(z) = \alpha (e^z - 1)$ if $z < 0$, else $z$")
plt.plot(z, selu(z), "r--", linewidth=2, label=r"SELU$(z) = 1.05 \, $ELU$_{1.67}(z)$")
plt.plot([-5, 5], [0, 0], 'k-')
plt.plot([-5, 5], [-1, -1], 'k:', linewidth=2)
plt.plot([-5, 5], [-1.758, -1.758], 'k:', linewidth=2)
plt.plot([0, 0], [-2.2, 3.2], 'k-')
plt.grid(True)
plt.axis([-5, 5, -2.2, 3.2])
plt.xlabel("$z$")
plt.gca().set_aspect("equal")
plt.legend()

save_fig("elu_selu_plot")
plt.show()
```



Sử dụng SELU rất đơn giản:

```
dense = tf.keras.layers.Dense(50, activation="selu",
                              kernel_initializer="lecun_normal")
```

Tài liệu bổ sung – một ví dụ về mạng được tự điều chuẩn (self-regularized network) sử dụng SELU

Hãy tạo một mạng nơ-ron cho Fashion MNIST với 100 lớp ẩn, sử dụng hàm kích hoạt SELU:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[28, 28]))
for layer in range(100):
    model.add(tf.keras.layers.Dense(100, activation="selu",
                                    kernel_initializer="lecun_normal"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`inp
  super().__init__(**kwargs)
```

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=tf.keras.optimizers.SGD(learning_rate=0.001),
              metrics=["accuracy"])
```

Bây giờ hãy huấn luyện nó. Đừng quên **chuẩn hóa (scale)** các đầu vào về giá trị trung bình 0 và độ lệch chuẩn 1:

```
fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
X_train, X_valid, X_test = X_train / 255, X_valid / 255, X_test / 255
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

```
pixel_means = X_train.mean(axis=0, keepdims=True)
pixel_stds = X_train.std(axis=0, keepdims=True)
X_train_scaled = (X_train - pixel_means) / pixel_stds
X_valid_scaled = (X_valid - pixel_means) / pixel_stds
X_test_scaled = (X_test - pixel_means) / pixel_stds
```

```
history = model.fit(X_train_scaled, y_train, epochs=5,
                    validation_data=(X_valid_scaled, y_valid))
```

```
Epoch 1/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 35s 17ms/step - accuracy: 0.5757 - loss: 1.1581 - val_accuracy: 0.7308 - val_loss: 0.7970
Epoch 2/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.7569 - loss: 0.7040 - val_accuracy: 0.7850 - val_loss: 0.6174
Epoch 3/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 28s 16ms/step - accuracy: 0.7901 - loss: 0.6088 - val_accuracy: 0.7902 - val_loss: 0.5966
Epoch 4/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 28s 16ms/step - accuracy: 0.7866 - loss: 0.6222 - val_accuracy: 0.8008 - val_loss: 0.5614
Epoch 5/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 27s 16ms/step - accuracy: 0.8238 - loss: 0.5077 - val_accuracy: 0.8020 - val_loss: 0.5305
```

Mạng đã **học** được, mặc dù nó **sâu** đến như vậy. Bây giờ hãy xem điều gì xảy ra nếu chúng ta thử sử dụng hàm kích hoạt **ReLU** thay thế:

```
tf.random.set_seed(42)
```

```
model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[28, 28]))
for layer in range(100):
    model.add(tf.keras.layers.Dense(100, activation="relu",
                                    kernel_initializer="he_normal"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=tf.keras.optimizers.SGD(learning_rate=0.001),
              metrics=["accuracy"])
```

```
history = model.fit(X_train_scaled, y_train, epochs=5,
                    validation_data=(X_valid_scaled, y_valid))
```

```
Epoch 1/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 36s 18ms/step - accuracy: 0.2856 - loss: 1.7623 - val_accuracy: 0.3946 - val_loss: 1.4030
Epoch 2/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 29s 17ms/step - accuracy: 0.5106 - loss: 1.1682 - val_accuracy: 0.6338 - val_loss: 0.9280
Epoch 3/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 29s 17ms/step - accuracy: 0.5550 - loss: 1.0963 - val_accuracy: 0.6188 - val_loss: 0.9924
Epoch 4/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 29s 17ms/step - accuracy: 0.6441 - loss: 0.8833 - val_accuracy: 0.6834 - val_loss: 0.8159
Epoch 5/5
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 30s 17ms/step - accuracy: 0.6893 - loss: 0.7683 - val_accuracy: 0.7172 - val_loss: 0.7038
```

Hoàn toàn không tốt chút nào, chúng ta đã gặp phải vấn đề **triệt tiêu/bùng nổ độ dốc**.

GELU, Swish và Mish

```
```

```
# extra code – this cell generates and saves Figure 11–4

def swish(z, beta=1):
    return z * sigmoid(beta * z)

def approx_gelu(z):
    return swish(z, beta=1.702)

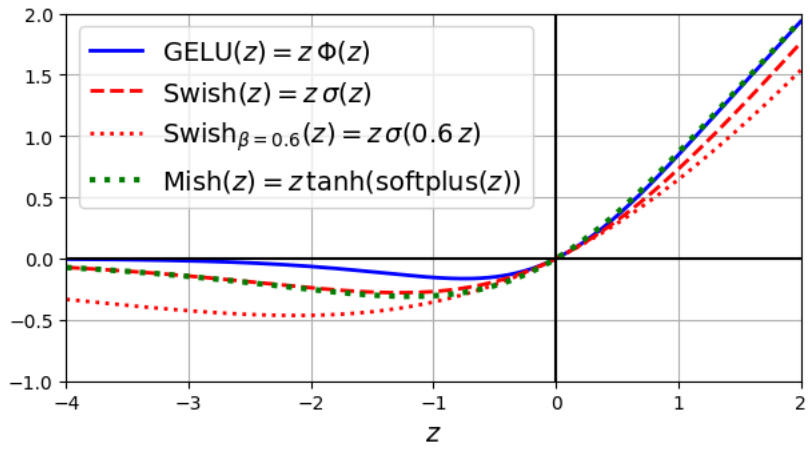
def softplus(z):
    return np.log(1 + np.exp(z))

def mish(z):
    return z * np.tanh(softplus(z))

z = np.linspace(-4, 2, 200)

beta = 0.6
plt.plot(z, approx_gelu(z), "b-", linewidth=2,
         label=r"GELU$(z) = z\,\Phi(z)$")
plt.plot(z, swish(z), "r--", linewidth=2,
         label=r"Swish$(z) = z\,\sigma(z)$")
plt.plot(z, swish(z, beta), "r:", linewidth=2,
         label=fr"Swish$_{{\beta={beta}}}(z)=z\,\sigma({beta}\,z)$")
plt.plot(z, mish(z), "g:", linewidth=3,
         label=fr"Mish$(z) = z\,\tanh($softplus$(z))$")
plt.plot([-4, 2], [0, 0], 'k-')
plt.plot([0, 0], [-2.2, 3.2], 'k-')
plt.grid(True)
plt.axis([-4, 2, -1, 2])
plt.gca().set_aspect("equal")
plt.xlabel("$z$")
plt.legend(loc="upper left")

save_fig("gelu_swish_mish_plot")
plt.show()
```



Chuẩn hóa Hàng loạt (Batch Normalization)

```
# extra code - clear the name counters and set the random seed
tf.keras.backend.clear_session()
tf.random.set_seed(42)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
batch_normalization (BatchNormalization)	(None, 784)	3,136
dense (Dense)	(None, 300)	235,500
batch_normalization_1 (BatchNormalization)	(None, 300)	1,200
dense_1 (Dense)	(None, 100)	30,100
batch_normalization_2 (BatchNormalization)	(None, 100)	400
dense_2 (Dense)	(None, 10)	1,010

Total params: 271,346 (1.04 MB)
Trainable params: 268,978 (1.03 MB)
Non-trainable params: 2,368 (9.25 KB)

```
[(var.name, var.trainable) for var in model.layers[1].variables]
```

```
[('gamma', True),
 ('beta', True),
 ('moving_mean', False),
 ('moving_variance', False)]
```

```
# extra code – just show that the model works! 😊
# model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
#               metrics="accuracy")
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=2, validation_data=(X_valid, y_valid))
```

```
Epoch 1/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 14s 7ms/step - accuracy: 0.8092 - loss: 0.5515 - val_accuracy: 0.8518 - val_loss: 0.4020
Epoch 2/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.8624 - loss: 0.3936 - val_accuracy: 0.8674 - val_loss: 0.3711
<keras.src.callbacks.history.History at 0x7e391b223110>
```

Đôi khi áp dụng **BN (Batch Normalization)** trước hàm kích hoạt hoạt động tốt hơn (có một cuộc tranh luận về chủ đề này). Hơn nữa, lớp đứng trước một lớp `BatchNormalization` không cần phải có các **hệ số thiên vị (bias terms)**, vì lớp `BatchNormalization` cũng có một số, điều đó sẽ là sự lãng phí tham số, do đó bạn có thể đặt `use_bias=False` khi tạo các lớp đó:

```
# extra code - clear the name counters and set the random seed
tf.keras.backend.clear_session()
tf.random.set_seed(42)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, kernel_initializer="he_normal", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(100, kernel_initializer="he_normal", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

```
# extra code – just show that the model works! 😊
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=2, validation_data=(X_valid, y_valid))
```

```
Epoch 1/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.8005 - loss: 0.5983 - val_accuracy: 0.8452 - val_loss: 0.4307
Epoch 2/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.8532 - loss: 0.4198 - val_accuracy: 0.8612 - val_loss: 0.3894
<keras.src.callbacks.history.History at 0x7e3918b3fa80>
```

Cắt bớt Độ dốc (Gradient Clipping)

Tất cả các `tf.keras.optimizers` đều chấp nhận các đối số `clipnorm` hoặc `clipvalue`:

```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer)
```

```
optimizer = tf.keras.optimizers.SGD(clipnorm=1.0)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer)
```

Tái sử dụng các Lớp đã được Huấn luyện trước (Reusing Pretrained Layers)

Tái sử dụng một mô hình Keras

Hãy chia tập huấn luyện fashion MNIST thành hai phần:

- `X_train_A`: tất cả các hình ảnh của tất cả các mặt hàng ngoại trừ áo phông/áo top và áo len chui đầu (các lớp 0 và 2).
- `X_train_B`: một tập huấn luyện nhỏ hơn nhiều chỉ gồm 200 hình ảnh đầu tiên của áo phông/áo top và áo len chui đầu.

Tập xác thực (validation set) và tập kiểm tra (test set) cũng được chia theo cách này, nhưng không giới hạn số lượng hình ảnh.

Chúng ta sẽ huấn luyện một mô hình trên tập A (nhiệm vụ phân loại với 8 lớp), và cố gắng tái sử dụng nó để giải quyết tập B (phân loại nhị phân). Chúng ta hy vọng truyền một chút kiến thức từ nhiệm vụ A sang nhiệm vụ B, vì các lớp trong tập A (quần dài, váy, áo khoác, sandal, áo sơ mi, giày thể thao, túi xách và bốt cổ chân) có phần nào tương tự với các lớp trong tập B (áo phông/áo top và áo len chui đầu). Tuy nhiên, vì chúng ta đang sử dụng các lớp `Dense`, chỉ những mẫu xuất hiện ở cùng một vị trí mới có thể được tái sử dụng (ngược lại, các lớp tích chập sẽ truyền tốt hơn nhiều, vì các mẫu đã học có thể được phát hiện ở bất cứ đâu trên hình ảnh, như chúng ta sẽ thấy trong chương 14).

```
# extra code – split Fashion MNIST into tasks A and B, then train and save
#              model A to "my_model_A".

pos_class_id = class_names.index("Pullover")
neg_class_id = class_names.index("T-shirt/top")

def split_dataset(X, y):
    y_for_B = (y == pos_class_id) | (y == neg_class_id)
    y_A = y[~y_for_B]
    y_B = (y[y_for_B] == pos_class_id).astype(np.float32)
    old_class_ids = list(set(range(10)) - set([neg_class_id, pos_class_id]))
    for old_class_id, new_class_id in zip(old_class_ids, range(8)):
        y_A[y_A == old_class_id] = new_class_id  # reorder class ids for A
    return ((X[~y_for_B], y_A), (X[y_for_B], y_B))

(X_train_A, y_train_A), (X_train_B, y_train_B) = split_dataset(X_train, y_train)
(X_valid_A, y_valid_A), (X_valid_B, y_valid_B) = split_dataset(X_valid, y_valid)
(X_test_A, y_test_A), (X_test_B, y_test_B) = split_dataset(X_test, y_test)
X_train_B = X_train_B[:200]
y_train_B = y_train_B[:200]

tf.random.set_seed(42)

model_A = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(8, activation="softmax")
])

model_A.compile(loss="sparse_categorical_crossentropy",
                optimizer=tf.keras.optimizers.SGD(learning_rate=0.001),
                metrics=["accuracy"])
history = model_A.fit(X_train_A, y_train_A, epochs=20,
                      validation_data=(X_valid_A, y_valid_A))
```

```
#model_A.save("my_model_A")
model_A.save("my_model_A.keras")
```

```
Epoch 1/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6269 - loss: 1.1908 - val_accuracy: 0.7806 - val_loss: 0.7075
Epoch 2/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8073 - loss: 0.6069 - val_accuracy: 0.8323 - val_loss: 0.5202
Epoch 3/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8429 - loss: 0.4878 - val_accuracy: 0.8483 - val_loss: 0.4450
Epoch 4/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.8586 - loss: 0.4284 - val_accuracy: 0.8629 - val_loss: 0.4023
Epoch 5/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8699 - loss: 0.3919 - val_accuracy: 0.8714 - val_loss: 0.3745
Epoch 6/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.8767 - loss: 0.3666 - val_accuracy: 0.8772 - val_loss: 0.3548
Epoch 7/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8818 - loss: 0.3479 - val_accuracy: 0.8804 - val_loss: 0.3401
Epoch 8/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8857 - loss: 0.3331 - val_accuracy: 0.8802 - val_loss: 0.3282
Epoch 9/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8895 - loss: 0.3210 - val_accuracy: 0.8822 - val_loss: 0.3186
Epoch 10/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8926 - loss: 0.3109 - val_accuracy: 0.8844 - val_loss: 0.3110
Epoch 11/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8956 - loss: 0.3022 - val_accuracy: 0.8869 - val_loss: 0.3042
Epoch 12/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8983 - loss: 0.2946 - val_accuracy: 0.8892 - val_loss: 0.2981
Epoch 13/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9006 - loss: 0.2878 - val_accuracy: 0.8925 - val_loss: 0.2924
Epoch 14/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9029 - loss: 0.2818 - val_accuracy: 0.8955 - val_loss: 0.2876
Epoch 15/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.9050 - loss: 0.2762 - val_accuracy: 0.8977 - val_loss: 0.2831
Epoch 16/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9070 - loss: 0.2712 - val_accuracy: 0.9010 - val_loss: 0.2790
Epoch 17/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9082 - loss: 0.2666 - val_accuracy: 0.9025 - val_loss: 0.2754
Epoch 18/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9094 - loss: 0.2624 - val_accuracy: 0.9060 - val_loss: 0.2720
Epoch 19/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.9112 - loss: 0.2585 - val_accuracy: 0.9065 - val_loss: 0.2690
Epoch 20/20
1376/1376 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9125 - loss: 0.2548 - val_accuracy: 0.9067 - val_loss: 0.2660
```

```
# extra code – train and evaluate model B, without reusing model A

tf.random.set_seed(42)
model_B = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model_B.compile(loss="binary_crossentropy",
                optimizer=tf.keras.optimizers.SGD(learning_rate=0.001),
                metrics=["accuracy"])
history = model_B.fit(X_train_B, y_train_B, epochs=20,
                      validation_data=(X_valid_B, y_valid_B))
model_B.evaluate(X_test_B, y_test_B)
```

```
Epoch 1/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - accuracy: 0.4450 - loss: 0.7166 - val_accuracy: 0.4876 - val_loss: 0.6613
Epoch 2/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4600 - loss: 0.6597 - val_accuracy: 0.5094 - val_loss: 0.6219
Epoch 3/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.5250 - loss: 0.6212 - val_accuracy: 0.6073 - val_loss: 0.5937
Epoch 4/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6500 - loss: 0.5929 - val_accuracy: 0.7112 - val_loss: 0.5724
Epoch 5/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.7150 - loss: 0.5703 - val_accuracy: 0.7725 - val_loss: 0.5550
Epoch 6/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8050 - loss: 0.5516 - val_accuracy: 0.8042 - val_loss: 0.5400
Epoch 7/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8300 - loss: 0.5353 - val_accuracy: 0.8259 - val_loss: 0.5268
Epoch 8/20
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8800 - loss: 0.5207 - val_accuracy: 0.8487 - val_loss: 0.5147
Epoch 9/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8900 - loss: 0.5073 - val_accuracy: 0.8655 - val_loss: 0.5032
Epoch 10/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9000 - loss: 0.4947 - val_accuracy: 0.8734 - val_loss: 0.4925
Epoch 11/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.9100 - loss: 0.4829 - val_accuracy: 0.8783 - val_loss: 0.4822
Epoch 12/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9100 - loss: 0.4717 - val_accuracy: 0.8882 - val_loss: 0.4724
Epoch 13/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9150 - loss: 0.4611 - val_accuracy: 0.9001 - val_loss: 0.4630
Epoch 14/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9150 - loss: 0.4509 - val_accuracy: 0.9031 - val_loss: 0.4539
Epoch 15/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9200 - loss: 0.4412 - val_accuracy: 0.9050 - val_loss: 0.4452
Epoch 16/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9200 - loss: 0.4320 - val_accuracy: 0.9120 - val_loss: 0.4367
Epoch 17/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - accuracy: 0.9250 - loss: 0.4230 - val_accuracy: 0.9159 - val_loss: 0.4285
Epoch 18/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.9300 - loss: 0.4144 - val_accuracy: 0.9189 - val_loss: 0.4207
Epoch 19/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.9350 - loss: 0.4061 - val_accuracy: 0.9238 - val_loss: 0.4130
Epoch 20/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.9350 - loss: 0.3981 - val_accuracy: 0.9268 - val_loss: 0.4057
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9040 - loss: 0.4173
[0.4172828197479248, 0.9039999842643738]
```

Mô hình B đạt độ chính xác 91.85% trên tập kiểm tra. Bây giờ hãy thử tái sử dụng mô hình A đã được huấn luyện trước.

```
#model_A = tf.keras.models.load_model("my_model_A")
model_A = tf.keras.models.load_model("my_model_A.keras")

model_B_on_A = tf.keras.Sequential(model_A.layers[:-1])
model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Lưu ý rằng `model_B_on_A` và `model_A` hiện tại thực sự **chia sẻ các lớp (share layers)**, vì vậy khi chúng ta huấn luyện một mô hình, nó sẽ cập nhật cả hai mô hình. Nếu chúng ta muốn tránh điều đó, chúng ta cần xây dựng `model_B_on_A` dựa trên một **bản sao (clone)** của `model_A`:

```
tf.random.set_seed(42)  # extra code – ensure reproducibility
```

```
model_A_clone = tf.keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

```
# extra code – creating model_B_on_A just like in the previous cell
model_B_on_A = tf.keras.Sequential(model_A_clone.layers[:-1])
model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
```

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B, y_valid_B))
```

```
Epoch 1/4
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - accuracy: 0.1300 - loss: 0.9580 - val_accuracy: 0.1494 - val_loss: 0.9218
Epoch 2/4
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.1300 - loss: 0.9352 - val_accuracy: 0.1612 - val_loss: 0.9032
Epoch 3/4
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.1300 - loss: 0.9144 - val_accuracy: 0.1721 - val_loss: 0.8854
Epoch 4/4
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.1450 - loss: 0.8947 - val_accuracy: 0.1860 - val_loss: 0.8683
Epoch 1/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - accuracy: 0.1900 - loss: 0.8463 - val_accuracy: 0.3867 - val_loss: 0.7813
Epoch 2/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.3600 - loss: 0.7571 - val_accuracy: 0.5084 - val_loss: 0.7086
Epoch 3/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.5550 - loss: 0.6826 - val_accuracy: 0.6063 - val_loss: 0.6505
Epoch 4/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.7100 - loss: 0.6227 - val_accuracy: 0.7280 - val_loss: 0.6021
Epoch 5/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.7900 - loss: 0.5732 - val_accuracy: 0.8101 - val_loss: 0.5615
Epoch 6/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8600 - loss: 0.5316 - val_accuracy: 0.8595 - val_loss: 0.5267
Epoch 7/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.9050 - loss: 0.4958 - val_accuracy: 0.8803 - val_loss: 0.4962
Epoch 8/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9250 - loss: 0.4646 - val_accuracy: 0.8971 - val_loss: 0.4698
Epoch 9/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9300 - loss: 0.4375 - val_accuracy: 0.9031 - val_loss: 0.4465
Epoch 10/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9350 - loss: 0.4137 - val_accuracy: 0.9100 - val_loss: 0.4259
Epoch 11/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9350 - loss: 0.3927 - val_accuracy: 0.9189 - val_loss: 0.4075
Epoch 12/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9450 - loss: 0.3742 - val_accuracy: 0.9228 - val_loss: 0.3911
Epoch 13/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.9500 - loss: 0.3574 - val_accuracy: 0.9268 - val_loss: 0.3763
Epoch 14/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9600 - loss: 0.3422 - val_accuracy: 0.9308 - val_loss: 0.3628
Epoch 15/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9600 - loss: 0.3284 - val_accuracy: 0.9347 - val_loss: 0.3503
Epoch 16/16
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9600 - loss: 0.3157 - val_accuracy: 0.9377 - val_loss: 0.3389
```

Vậy, **phán quyết cuối cùng** là gì?

```
model_B_on_A.evaluate(X_test_B, y_test_B)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9210 - loss: 0.3420
[0.3420190215110779, 0.9210000038146973]
```

Tuyệt vời! Chúng ta đã có được một chút **chuyển giao (transfer)**: độ chính xác của mô hình đã tăng thêm 2 điểm phần trăm, từ 91.85% lên 93.85%. Điều này có nghĩa là **tỷ lệ lỗi (error rate)** đã giảm gần 25%:

```
1 - (100 - 93.85) / (100 - 91.85)
```

```
0.24539877300613477
```

Các Trình Tối ưu hóa Nhanh hơn (Faster Optimizers)

```
# extra code – a little function to test an optimizer on Fashion MNIST

def build_model(seed=42):
    tf.random.set_seed(seed)
    return tf.keras.Sequential([
        tf.keras.layers.Flatten(input_shape=[28, 28]),
        tf.keras.layers.Dense(100, activation="relu",
                              kernel_initializer="he_normal"),
        tf.keras.layers.Dense(100, activation="relu",
                              kernel_initializer="he_normal"),
        tf.keras.layers.Dense(100, activation="relu",
                              kernel_initializer="he_normal"),
        tf.keras.layers.Dense(10, activation="softmax")
    ])

def build_and_train_model(optimizer):
    model = build_model()
    model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
                  metrics=["accuracy"])
    return model.fit(X_train, y_train, epochs=10,
                     validation_data=(X_valid, y_valid))
```

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)
```

```
history_sgd = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.7539 - loss: 0.7204 - val_accuracy: 0.8240 - val_loss: 0.5039
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8343 - loss: 0.4720 - val_accuracy: 0.8374 - val_loss: 0.4518
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8502 - loss: 0.4263 - val_accuracy: 0.8518 - val_loss: 0.4203
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8585 - loss: 0.3985 - val_accuracy: 0.8552 - val_loss: 0.4019
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8662 - loss: 0.3778 - val_accuracy: 0.8604 - val_loss: 0.3905
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8713 - loss: 0.3613 - val_accuracy: 0.8616 - val_loss: 0.3804
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8753 - loss: 0.3472 - val_accuracy: 0.8646 - val_loss: 0.3723
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8791 - loss: 0.3353 - val_accuracy: 0.8648 - val_loss: 0.3659
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8829 - loss: 0.3250 - val_accuracy: 0.8672 - val_loss: 0.3609
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8863 - loss: 0.3156 - val_accuracy: 0.8676 - val_loss: 0.3548
```

Tối ưu hóa Momentum

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)
```

```
history_momentum = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7577 - loss: 0.7046 - val_accuracy: 0.8182 - val_loss: 0.5115
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8347 - loss: 0.4685 - val_accuracy: 0.8348 - val_loss: 0.4562
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8494 - loss: 0.4241 - val_accuracy: 0.8466 - val_loss: 0.4239
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8591 - loss: 0.3963 - val_accuracy: 0.8506 - val_loss: 0.4059
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8655 - loss: 0.3759 - val_accuracy: 0.8544 - val_loss: 0.3917
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8716 - loss: 0.3593 - val_accuracy: 0.8572 - val_loss: 0.3832
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8760 - loss: 0.3455 - val_accuracy: 0.8572 - val_loss: 0.3786
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8798 - loss: 0.3341 - val_accuracy: 0.8572 - val_loss: 0.3744
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8826 - loss: 0.3239 - val_accuracy: 0.8584 - val_loss: 0.3706
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8856 - loss: 0.3151 - val_accuracy: 0.8584 - val_loss: 0.3666
```

Độ dốc Tăng tốc Nesterov (Nesterov Accelerated Gradient)

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9,
                                    nesterov=True)
```

```
history_nesterov = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.7713 - loss: 0.6787 - val_accuracy: 0.8226 - val_loss: 0.4946
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8379 - loss: 0.4612 - val_accuracy: 0.8378 - val_loss: 0.4452
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8518 - loss: 0.4180 - val_accuracy: 0.8444 - val_loss: 0.4234
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8617 - loss: 0.3916 - val_accuracy: 0.8486 - val_loss: 0.4086
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8688 - loss: 0.3720 - val_accuracy: 0.8510 - val_loss: 0.3971
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8731 - loss: 0.3565 - val_accuracy: 0.8562 - val_loss: 0.3866
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8782 - loss: 0.3432 - val_accuracy: 0.8574 - val_loss: 0.3824
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8817 - loss: 0.3320 - val_accuracy: 0.8584 - val_loss: 0.3767
```

```
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8849 - loss: 0.3220 - val_accuracy: 0.8572 - val_loss: 0.3738
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8882 - loss: 0.3131 - val_accuracy: 0.8578 - val_loss: 0.3694
```

AdaGrad

```
optimizer = tf.keras.optimizers.Adagrad(learning_rate=0.001)
```

```
history_adagrad = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.6932 - loss: 0.9791 - val_accuracy: 0.7774 - val_loss: 0.6637
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.7957 - loss: 0.6234 - val_accuracy: 0.8102 - val_loss: 0.5669
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8150 - loss: 0.5571 - val_accuracy: 0.8220 - val_loss: 0.5239
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8247 - loss: 0.5227 - val_accuracy: 0.8292 - val_loss: 0.4992
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8313 - loss: 0.5008 - val_accuracy: 0.8348 - val_loss: 0.4823
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8364 - loss: 0.4850 - val_accuracy: 0.8372 - val_loss: 0.4698
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8393 - loss: 0.4728 - val_accuracy: 0.8396 - val_loss: 0.4599
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8425 - loss: 0.4629 - val_accuracy: 0.8414 - val_loss: 0.4520
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8452 - loss: 0.4545 - val_accuracy: 0.8420 - val_loss: 0.4453
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8468 - loss: 0.4474 - val_accuracy: 0.8446 - val_loss: 0.4395
```

RMSProp

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

```
history_rmsprop = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8131 - loss: 0.5165 - val_accuracy: 0.8438 - val_loss: 0.4080
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8589 - loss: 0.3904 - val_accuracy: 0.8472 - val_loss: 0.4170
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8697 - loss: 0.3637 - val_accuracy: 0.8426 - val_loss: 0.4589
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8751 - loss: 0.3524 - val_accuracy: 0.8448 - val_loss: 0.4565
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8786 - loss: 0.3481 - val_accuracy: 0.8484 - val_loss: 0.4756
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8805 - loss: 0.3446 - val_accuracy: 0.8568 - val_loss: 0.4471
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8824 - loss: 0.3431 - val_accuracy: 0.8510 - val_loss: 0.4915
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8839 - loss: 0.3413 - val_accuracy: 0.8530 - val_loss: 0.4686
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8849 - loss: 0.3399 - val_accuracy: 0.8600 - val_loss: 0.4584
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8863 - loss: 0.3385 - val_accuracy: 0.8598 - val_loss: 0.5172
```

Tối ưu hóa Adam

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9,
                                     beta_2=0.999)
```

```
history_adam = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8201 - loss: 0.5001 - val_accuracy: 0.8328 - val_loss: 0.4320
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8639 - loss: 0.3730 - val_accuracy: 0.8504 - val_loss: 0.3970
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8757 - loss: 0.3372 - val_accuracy: 0.8584 - val_loss: 0.3734
Epoch 4/10
```

```
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8849 - loss: 0.3121 - val_accuracy: 0.8608 - val_loss: 0.3796
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8892 - loss: 0.2966 - val_accuracy: 0.8662 - val_loss: 0.3616
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8956 - loss: 0.2801 - val_accuracy: 0.8726 - val_loss: 0.3635
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8985 - loss: 0.2695 - val_accuracy: 0.8748 - val_loss: 0.3681
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9035 - loss: 0.2561 - val_accuracy: 0.8666 - val_loss: 0.3898
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9064 - loss: 0.2484 - val_accuracy: 0.8702 - val_loss: 0.3892
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9094 - loss: 0.2370 - val_accuracy: 0.8788 - val_loss: 0.3958
```

Tối ưu hóa Adamax

```
optimizer = tf.keras.optimizers.Adamax(learning_rate=0.001, beta_1=0.9,
                                       beta_2=0.999)
```

```
history_adamax = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8113 - loss: 0.5391 - val_accuracy: 0.8316 - val_loss: 0.4421
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8577 - loss: 0.3968 - val_accuracy: 0.8434 - val_loss: 0.4083
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8714 - loss: 0.3558 - val_accuracy: 0.8500 - val_loss: 0.3940
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8801 - loss: 0.3311 - val_accuracy: 0.8586 - val_loss: 0.3734
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8865 - loss: 0.3122 - val_accuracy: 0.8646 - val_loss: 0.3623
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8912 - loss: 0.2971 - val_accuracy: 0.8634 - val_loss: 0.3654
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8959 - loss: 0.2848 - val_accuracy: 0.8660 - val_loss: 0.3590
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8995 - loss: 0.2734 - val_accuracy: 0.8660 - val_loss: 0.3602
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9029 - loss: 0.2634 - val_accuracy: 0.8684 - val_loss: 0.3543
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9068 - loss: 0.2537 - val_accuracy: 0.8668 - val_loss: 0.3639
```

Tối ưu hóa Nadam

```
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.001, beta_1=0.9,
                                      beta_2=0.999)
```

```
history_nadam = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8296 - loss: 0.4818 - val_accuracy: 0.8372 - val_loss: 0.4207
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8665 - loss: 0.3651 - val_accuracy: 0.8420 - val_loss: 0.4102
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8793 - loss: 0.3294 - val_accuracy: 0.8562 - val_loss: 0.3812
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8864 - loss: 0.3059 - val_accuracy: 0.8544 - val_loss: 0.3857
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8923 - loss: 0.2885 - val_accuracy: 0.8608 - val_loss: 0.3903
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8975 - loss: 0.2723 - val_accuracy: 0.8590 - val_loss: 0.4011
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9032 - loss: 0.2584 - val_accuracy: 0.8640 - val_loss: 0.3756
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9075 - loss: 0.2478 - val_accuracy: 0.8738 - val_loss: 0.3691
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9098 - loss: 0.2379 - val_accuracy: 0.8692 - val_loss: 0.3626
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9150 - loss: 0.2286 - val_accuracy: 0.8772 - val_loss: 0.3786
```

Tối ưu hóa AdamW

Lưu ý: Kể từ TF 1.12, `AdamW` không còn là thử nghiệm nữa. Nó có sẵn tại `tf.keras.optimizers.AdamW` thay vì `tf.keras.optimizers.experimental.AdamW`.

```
optimizer = tf.keras.optimizers.AdamW(weight_decay=1e-5, learning_rate=0.001,
                                      beta_1=0.9, beta_2=0.999)
```

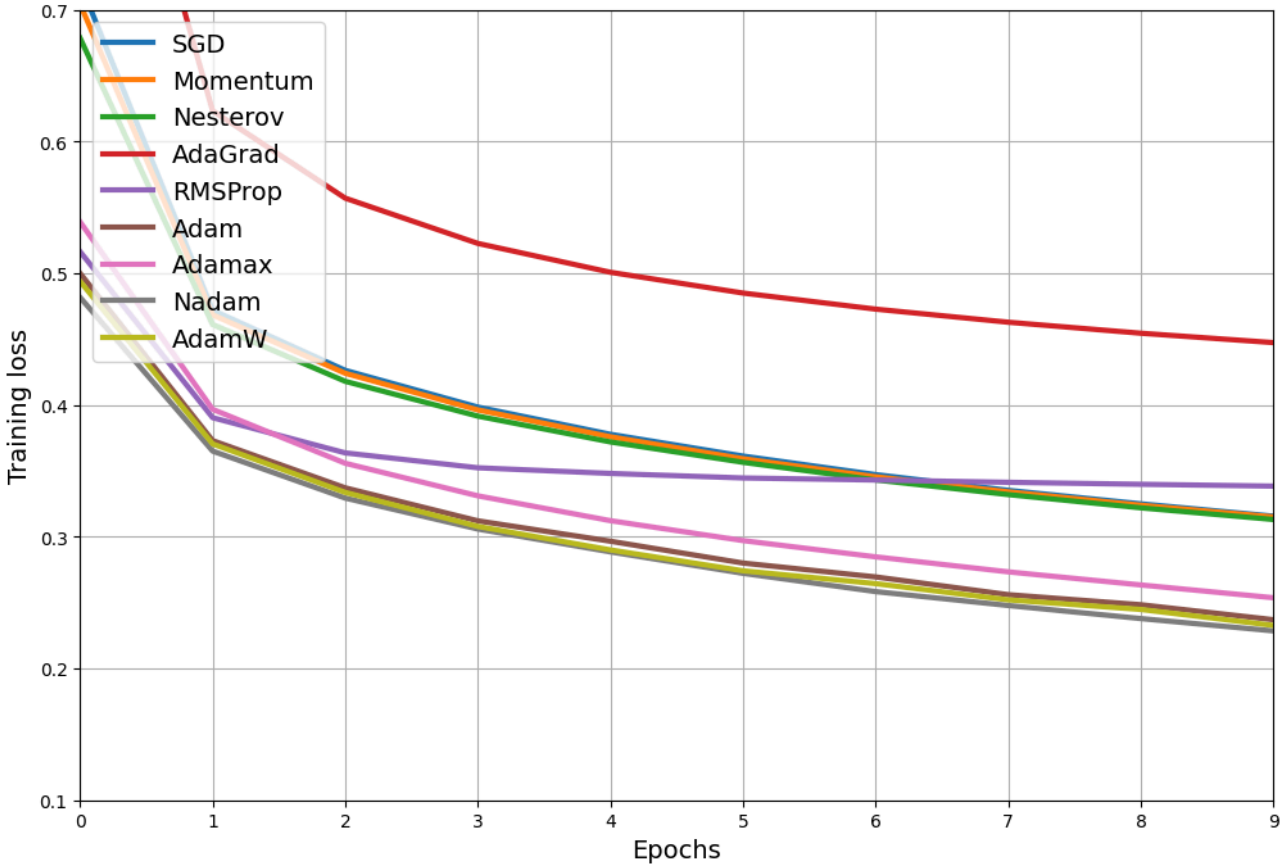
```
history_adamw = build_and_train_model(optimizer)  # extra code
```

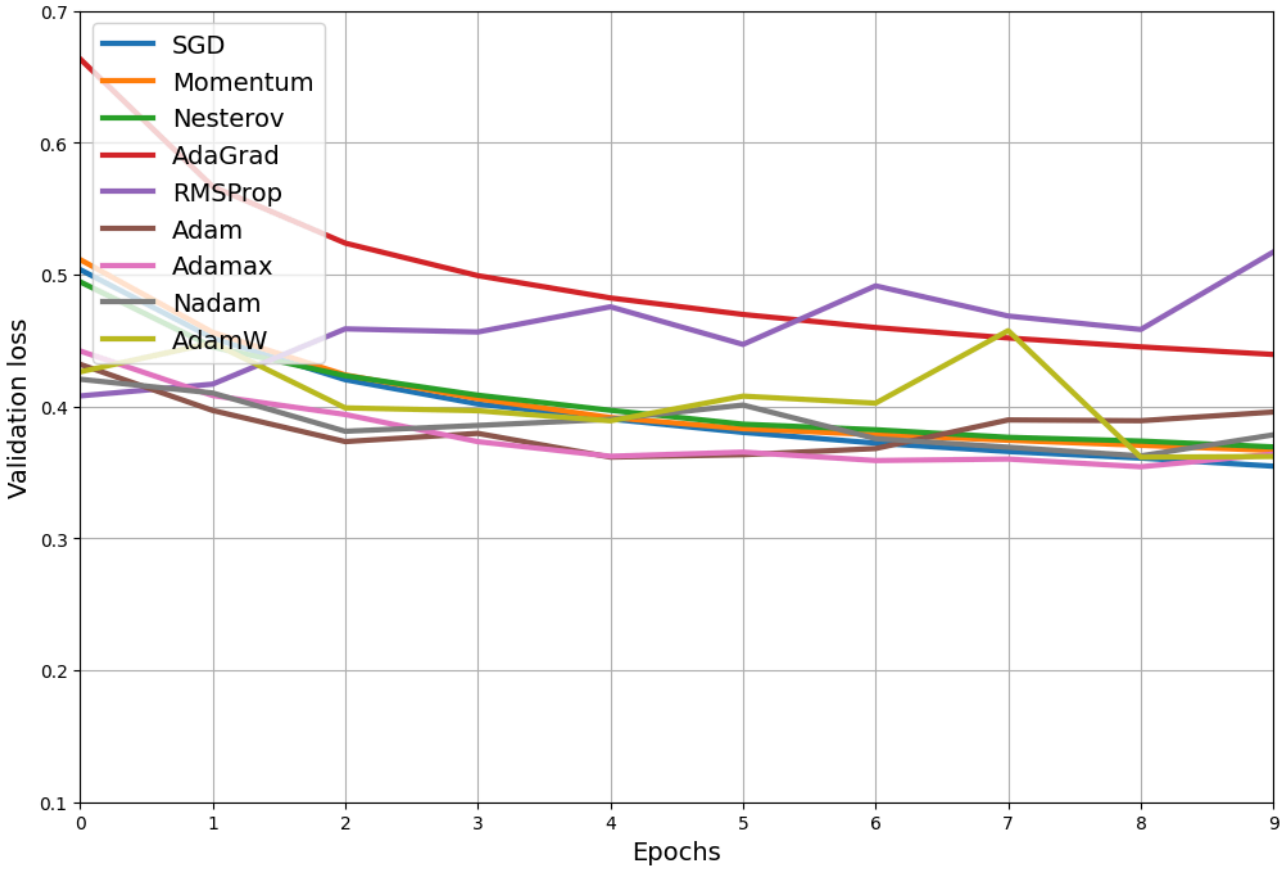
```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8231 - loss: 0.4940 - val_accuracy: 0.8332 - val_loss: 0.4263
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8646 - loss: 0.3705 - val_accuracy: 0.8196 - val_loss: 0.4483
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8775 - loss: 0.3337 - val_accuracy: 0.8490 - val_loss: 0.3990
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8860 - loss: 0.3079 - val_accuracy: 0.8560 - val_loss: 0.3968
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8920 - loss: 0.2900 - val_accuracy: 0.8598 - val_loss: 0.3892
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8976 - loss: 0.2742 - val_accuracy: 0.8556 - val_loss: 0.4079
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9006 - loss: 0.2644 - val_accuracy: 0.8574 - val_loss: 0.4026
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9058 - loss: 0.2521 - val_accuracy: 0.8528 - val_loss: 0.4575
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9086 - loss: 0.2448 - val_accuracy: 0.8748 - val_loss: 0.3616
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9122 - loss: 0.2327 - val_accuracy: 0.8690 - val_loss: 0.3621
```

```
# extra code – visualize the learning curves of all the optimizers

for loss in ("loss", "val_loss"):
    plt.figure(figsize=(12, 8))
    opt_names = "SGD Momentum Nesterov AdaGrad RMSProp Adam Adamax Nadam AdamW"
    for history, opt_name in zip((history_sgd, history_momentum, history_nesterov,
                                  history_adagrad, history_rmsprop, history_adam,
                                  history_adamax, history_nadam, history_adamw),
                                 opt_names.split()):
        plt.plot(history.history[loss], label=f"{opt_name}", linewidth=3)

    plt.grid()
    plt.xlabel("Epochs")
    plt.ylabel({"loss": "Training loss", "val_loss": "Validation loss"}[loss])
    plt.legend(loc="upper left")
    plt.axis([0, 9, 0.1, 0.7])
    plt.show()
```





Lập lịch Tốc độ Học (Learning Rate Scheduling)

Lập lịch Theo Hàm Mũ (Power Scheduling)

```
learning_rate = initial_learning_rate / (1 + step / decay_steps)**power
```

Keras dùng `power = 1`.

```
Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).
```

Lưu ý: Đối số `decay` trong các **trình tối ưu hóa (optimizers)** đã bị **khai tử (deprecated)**. Các trình tối ưu hóa cũ có triển khai đối số `decay` vẫn có sẵn trong `tf.keras.optimizers.legacy`, nhưng thay vào đó, bạn nên sử dụng các **bộ lập lịch (schedulers)** trong `tf.keras.optimizers.schedules`.

```
# DEPRECATED:
#optimizer = tf.keras.optimizers.legacy.SGD(learning_rate=0.01, decay=1e-4)
# RECOMMENDED:
lr_schedule = tf.keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=0.01,
    decay_steps=10_000,
    decay_rate=1.0,
    staircase=False
)
optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
```

```
# RECOMMENDED:
lr_schedule = tf.keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=0.01,
    decay_steps=10_000,
    decay_rate=1.0,
    staircase=False
)
optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
```

Bộ lập lịch `InverseTimeDecay` sử dụng công thức `learning_rate = initial_learning_rate / (1 + decay_rate * step / decay_steps)`. Nếu bạn đặt `staircase=True`, thì nó sẽ thay thế `step / decay_step` bằng `floor(step / decay_step)`.

```
history_power_scheduling = build_and_train_model(optimizer)  # extra code
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7584 - loss: 0.6988 - val_accuracy: 0.8300 - val_loss: 0.4822
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8327 - loss: 0.4751 - val_accuracy: 0.8434 - val_loss: 0.4366
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8487 - loss: 0.4290 - val_accuracy: 0.8510 - val_loss: 0.4139
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8577 - loss: 0.4029 - val_accuracy: 0.8560 - val_loss: 0.3986
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8634 - loss: 0.3850 - val_accuracy: 0.8590 - val_loss: 0.3863
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - accuracy: 0.8680 - loss: 0.3717 - val_accuracy: 0.8644 - val_loss: 0.3778
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8713 - loss: 0.3610 - val_accuracy: 0.8658 - val_loss: 0.3718
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8746 - loss: 0.3523 - val_accuracy: 0.8676 - val_loss: 0.3671
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8769 - loss: 0.3449 - val_accuracy: 0.8688 - val_loss: 0.3623
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8793 - loss: 0.3385 - val_accuracy: 0.8694 - val_loss: 0.3589
```

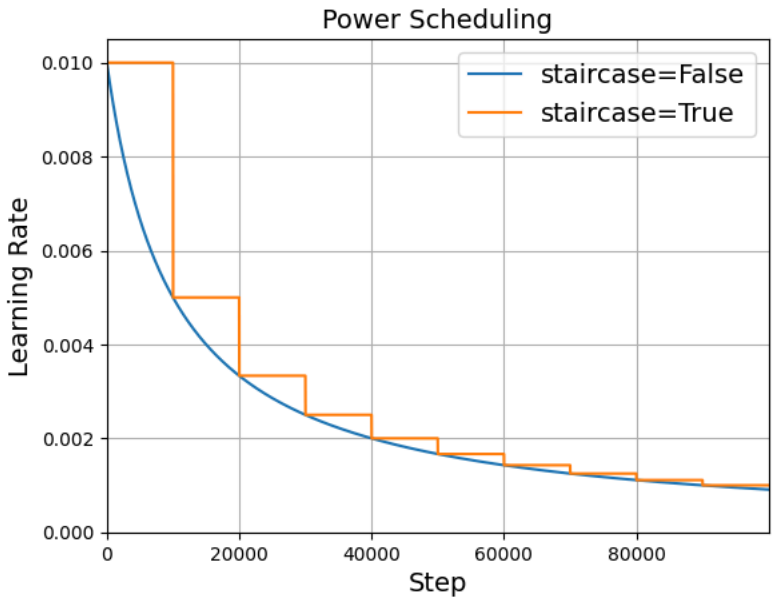
```
# extra code – this cell plots power scheduling with staircase=True or False

initial_learning_rate = 0.01
decay_rate = 1.0
decay_steps = 10_000

steps = np.arange(100_000)
```

```
lrs = initial_learning_rate / (1 + decay_rate * steps / decay_steps)
lrs2 = initial_learning_rate / (1 + decay_rate * np.floor(steps / decay_steps))

plt.plot(steps, lrs,  "-", label="staircase=False")
plt.plot(steps, lrs2,  "-", label="staircase=True")
plt.axis([0, steps.max(), 0, 0.0105])
plt.xlabel("Step")
plt.ylabel("Learning Rate")
plt.title("Power Scheduling", fontsize=14)
plt.legend()
plt.grid(True)
plt.show()
```



Lập lịch Theo Hàm Số Mũ (Exponential Scheduling)

```
learning_rate = initial_learning_rate * decay_rate ** (step / decay_steps)
```

```
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.01,
    decay_steps=20_000,
    decay_rate=0.1,
    staircase=False
)
optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
```

```
history_exponential_scheduling = build_and_train_model(optimizer)  # extra code
```

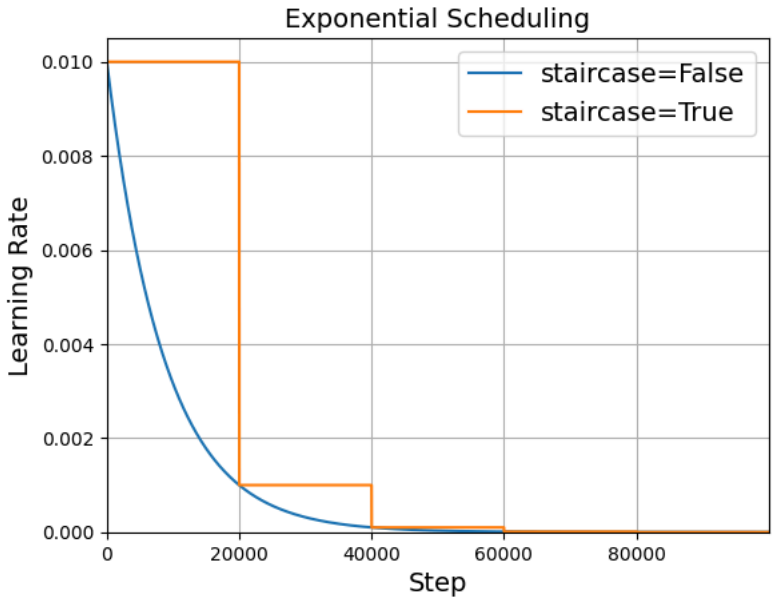
```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7658 - loss: 0.6832 - val_accuracy: 0.8330 - val_loss: 0.4839
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8360 - loss: 0.4672 - val_accuracy: 0.8406 - val_loss: 0.4391
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8518 - loss: 0.4233 - val_accuracy: 0.8484 - val_loss: 0.4140
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8601 - loss: 0.3986 - val_accuracy: 0.8550 - val_loss: 0.3988
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 11s 4ms/step - accuracy: 0.8663 - loss: 0.3820 - val_accuracy: 0.8616 - val_loss: 0.3875
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8698 - loss: 0.3701 - val_accuracy: 0.8646 - val_loss: 0.3792
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8724 - loss: 0.3611 - val_accuracy: 0.8674 - val_loss: 0.3738
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8748 - loss: 0.3542 - val_accuracy: 0.8694 - val_loss: 0.3693
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8768 - loss: 0.3487 - val_accuracy: 0.8708 - val_loss: 0.3658
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8781 - loss: 0.3443 - val_accuracy: 0.8712 - val_loss: 0.3630
```

```
# extra code – this cell plots exponential scheduling

initial_learning_rate = 0.01
decay_rate = 0.1
decay_steps = 20_000

steps = np.arange(100_000)
lrs = initial_learning_rate * decay_rate ** (steps / decay_steps)
lrs2 = initial_learning_rate * decay_rate ** np.floor(steps / decay_steps)

plt.plot(steps, lrs,  "-", label="staircase=False")
plt.plot(steps, lrs2,  "-", label="staircase=True")
plt.axis([0, steps.max(), 0, 0.0105])
plt.xlabel("Step")
plt.ylabel("Learning Rate")
plt.title("Exponential Scheduling", fontsize=14)
plt.legend()
plt.grid(True)
plt.show()
```



Keras cũng cung cấp lớp callback `LearningRateScheduler` cho phép bạn định nghĩa hàm lập lịch của riêng mình. Hãy xem cách bạn có thể sử dụng nó để triển khai **phân rã hàm số mũ (exponential decay)**. Lưu ý rằng trong trường hợp này, tốc độ học **chỉ thay đổi sau mỗi epoch**, chứ không phải sau mỗi bước:

```
def exponential_decay_fn(epoch):
    return 0.01 * 0.1 ** (epoch / 20)
```

```
def exponential_decay(lr0, s):
    def exponential_decay_fn(epoch):
        return lr0 * 0.1 ** (epoch / s)
    return exponential_decay_fn

exponential_decay_fn = exponential_decay(lr0=0.01, s=20)
```

```
# extra code – build and compile a model for Fashion MNIST

tf.random.set_seed(42)
model = build_model()
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
```

```
n_epochs = 20

lr_scheduler = tf.keras.callbacks.LearningRateScheduler(exponential_decay_fn)
history = model.fit(X_train, y_train, epochs=n_epochs,
```

```
                    validation_data=(X_valid, y_valid),
                    callbacks=[lr_scheduler])
```

```
Epoch 1/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7616 - loss: 0.6927 - val_accuracy: 0.8286 - val_loss: 0.4851 - learnin
Epoch 2/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8334 - loss: 0.4717 - val_accuracy: 0.8372 - val_loss: 0.4460 - learnin
Epoch 3/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8495 - loss: 0.4273 - val_accuracy: 0.8418 - val_loss: 0.4291 - learnin
Epoch 4/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8585 - loss: 0.4013 - val_accuracy: 0.8444 - val_loss: 0.4155 - learnin
Epoch 5/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8645 - loss: 0.3830 - val_accuracy: 0.8490 - val_loss: 0.4048 - learnin
Epoch 6/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - accuracy: 0.8690 - loss: 0.3687 - val_accuracy: 0.8544 - val_loss: 0.3970 - learnin
Epoch 7/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8729 - loss: 0.3572 - val_accuracy: 0.8570 - val_loss: 0.3887 - learnin
Epoch 8/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8760 - loss: 0.3477 - val_accuracy: 0.8586 - val_loss: 0.3822 - learnin
Epoch 9/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8788 - loss: 0.3398 - val_accuracy: 0.8606 - val_loss: 0.3762 - learnin
Epoch 10/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8813 - loss: 0.3330 - val_accuracy: 0.8646 - val_loss: 0.3701 - learni
Epoch 11/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8830 - loss: 0.3272 - val_accuracy: 0.8660 - val_loss: 0.3652 - learnin
Epoch 12/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8846 - loss: 0.3222 - val_accuracy: 0.8676 - val_loss: 0.3613 - learnin
Epoch 13/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8860 - loss: 0.3179 - val_accuracy: 0.8708 - val_loss: 0.3576 - learnin
Epoch 14/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8869 - loss: 0.3140 - val_accuracy: 0.8720 - val_loss: 0.3546 - learnin
Epoch 15/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8883 - loss: 0.3107 - val_accuracy: 0.8726 - val_loss: 0.3518 - learnin
Epoch 16/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8895 - loss: 0.3078 - val_accuracy: 0.8728 - val_loss: 0.3498 - learnin
Epoch 17/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8904 - loss: 0.3052 - val_accuracy: 0.8730 - val_loss: 0.3476 - learnin
Epoch 18/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8912 - loss: 0.3029 - val_accuracy: 0.8742 - val_loss: 0.3458 - learnin
Epoch 19/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - accuracy: 0.8920 - loss: 0.3009 - val_accuracy: 0.8748 - val_loss: 0.3441 - learni
Epoch 20/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8924 - loss: 0.2990 - val_accuracy: 0.8756 - val_loss: 0.3427 - learnin
```

Hoặc, hàm lập lịch có thể nhận **tốc độ học hiện tại** làm đối số thứ hai:

```
def exponential_decay_fn(epoch, lr):
    return lr * 0.1 ** (1 / 20)
```

Tài liệu bổ sung: nếu bạn muốn sử dụng một hàm lập lịch tùy chỉnh cập nhật **tốc độ học** sau **mỗi lần lặp (iteration)** thay vì sau **mỗi epoch**, bạn có thể viết lớp **callback** của riêng mình như sau:

```
# K = tf.keras.backend

# class ExponentialDecay(tf.keras.callbacks.Callback):
#     def __init__(self, n_steps=40_000):
#         super().__init__()
#         self.n_steps = n_steps

#     def on_batch_begin(self, batch, logs=None):
#         # Note: the `batch` argument is reset at each epoch
#         lr = K.get_value(self.model.optimizer.learning_rate)
#         new_learning_rate = lr * 0.1 ** (1 / self.n_steps)
#         K.set_value(self.model.optimizer.learning_rate, new_learning_rate)

#     def on_epoch_end(self, epoch, logs=None):
#         logs = logs or {}
#         logs['lr'] = K.get_value(self.model.optimizer.learning_rate)
import tensorflow as tf
from tensorflow.keras import backend as K

class CustomLRScheduler(tf.keras.callbacks.Callback):
    def __init__(self, n_steps=40000, decay_factor=0.1):
        super().__init__()
        self.n_steps = n_steps
        self.decay_factor = decay_factor
```

```
    def on_batch_begin(self, batch, logs=None):
        lr = float(self.model.optimizer.learning_rate.numpy())
        new_lr = lr * (self.decay_factor ** (1 / self.n_steps))
        self.model.optimizer.learning_rate.assign(new_lr)  # ✅ dùng assign thay vì K.set_value

    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        logs['learning_rate'] = float(self.model.optimizer.learning_rate.numpy())
```

```
# ✅ Tạo model mới
import tensorflow as tf
from tensorflow.keras import layers, models
model = models.Sequential([
    layers.Flatten(input_shape=[28, 28]),
    layers.Dense(100, activation="relu"),
    layers.Dense(100, activation="relu"),
    layers.Dense(10, activation="softmax")
])

# ✅ compile với optimizer object
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=optimizer,
    metrics=["accuracy"]
)

n_epochs = 20
batch_size = 32
n_steps = n_epochs * (len(X_train) // batch_size)
lr_scheduler = CustomLRScheduler(n_steps=n_steps)

history = model.fit(
    X_train, y_train,
    epochs=n_epochs,
    validation_data=(X_valid, y_valid),
    callbacks=[lr_scheduler]
)
```

```
Epoch 1/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7345 - loss: 0.7831 - val_accuracy: 0.8158 - val_loss: 0.5313 - learnin
Epoch 2/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8236 - loss: 0.5091 - val_accuracy: 0.8330 - val_loss: 0.4726 - learnin
Epoch 3/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8390 - loss: 0.4632 - val_accuracy: 0.8394 - val_loss: 0.4478 - learnin
Epoch 4/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8483 - loss: 0.4379 - val_accuracy: 0.8466 - val_loss: 0.4308 - learnin
Epoch 5/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8545 - loss: 0.4206 - val_accuracy: 0.8506 - val_loss: 0.4198 - learnin
Epoch 6/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8586 - loss: 0.4075 - val_accuracy: 0.8542 - val_loss: 0.4103 - learnin
Epoch 7/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8619 - loss: 0.3973 - val_accuracy: 0.8582 - val_loss: 0.4026 - learnin
Epoch 8/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8644 - loss: 0.3890 - val_accuracy: 0.8590 - val_loss: 0.3962 - learnin
Epoch 9/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8669 - loss: 0.3822 - val_accuracy: 0.8604 - val_loss: 0.3907 - learnin
Epoch 10/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8689 - loss: 0.3764 - val_accuracy: 0.8612 - val_loss: 0.3860 - learnin
Epoch 11/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8708 - loss: 0.3715 - val_accuracy: 0.8620 - val_loss: 0.3819 - learnin
Epoch 12/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8723 - loss: 0.3672 - val_accuracy: 0.8626 - val_loss: 0.3785 - learnin
Epoch 13/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8738 - loss: 0.3635 - val_accuracy: 0.8632 - val_loss: 0.3756 - learnin
Epoch 14/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8747 - loss: 0.3603 - val_accuracy: 0.8640 - val_loss: 0.3730 - learni
Epoch 15/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8757 - loss: 0.3575 - val_accuracy: 0.8652 - val_loss: 0.3707 - learnin
Epoch 16/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8762 - loss: 0.3551 - val_accuracy: 0.8658 - val_loss: 0.3686 - learnin
Epoch 17/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8769 - loss: 0.3529 - val_accuracy: 0.8658 - val_loss: 0.3668 - learnin
Epoch 18/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8772 - loss: 0.3510 - val_accuracy: 0.8678 - val_loss: 0.3652 - learnin
Epoch 19/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8782 - loss: 0.3494 - val_accuracy: 0.8688 - val_loss: 0.3638 - learnin
```

```
Epoch 20/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8786 - loss: 0.3479 - val_accuracy: 0.8698 - val_loss: 0.3625 - learnin
```

callback tương thích với Keras 3

```
import math
import tensorflow as tf

class ExponentialDecay(tf.keras.callbacks.Callback):
    def __init__(self, n_steps=40_000):
        super().__init__()
        self.n_steps = n_steps

    def on_batch_begin(self, batch, logs=None):
        # ✅ Lấy learning_rate hiện tại
        lr = float(self.model.optimizer.learning_rate.numpy())
        # ✅ Tính LR mới
        new_learning_rate = lr * 0.1 ** (1 / self.n_steps)
        # ✅ Gán lại giá trị LR
        self.model.optimizer.learning_rate.assign(new_learning_rate)

    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        logs['lr'] = float(self.model.optimizer.learning_rate.numpy())
```

Compile model đúng cách

```
# KHÔNG được dùng optimizer="sgd"
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=optimizer,
    metrics=["accuracy"]
)
```

Huấn luyện

```
batch_size = 32
n_epochs = 20
n_steps = n_epochs * math.ceil(len(X_train) / batch_size)
exp_decay = ExponentialDecay(n_steps)

history = model.fit(
    X_train, y_train,
    epochs=n_epochs,
    validation_data=(X_valid, y_valid),
    callbacks=[exp_decay]
)
```

```
Epoch 1/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8707 - loss: 0.3658 - val_accuracy: 0.8546 - val_loss: 0.3919 - lr: 0.0
Epoch 2/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8753 - loss: 0.3535 - val_accuracy: 0.8580 - val_loss: 0.3823 - lr: 0.0
Epoch 3/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8784 - loss: 0.3439 - val_accuracy: 0.8622 - val_loss: 0.3736 - lr: 0.0
Epoch 4/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8807 - loss: 0.3359 - val_accuracy: 0.8662 - val_loss: 0.3668 - lr: 0.0
Epoch 5/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8831 - loss: 0.3291 - val_accuracy: 0.8690 - val_loss: 0.3608 - lr: 0.0
Epoch 6/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8850 - loss: 0.3232 - val_accuracy: 0.8684 - val_loss: 0.3565 - lr: 0.0
Epoch 7/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8867 - loss: 0.3182 - val_accuracy: 0.8692 - val_loss: 0.3521 - lr: 0.
Epoch 8/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8881 - loss: 0.3138 - val_accuracy: 0.8724 - val_loss: 0.3487 - lr: 0.0
Epoch 9/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8895 - loss: 0.3099 - val_accuracy: 0.8740 - val_loss: 0.3460 - lr: 0.0
Epoch 10/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8907 - loss: 0.3066 - val_accuracy: 0.8752 - val_loss: 0.3436 - lr: 0.0
Epoch 11/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8916 - loss: 0.3036 - val_accuracy: 0.8764 - val_loss: 0.3415 - lr: 0.0
```

```
Epoch 12/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8927 - loss: 0.3010 - val_accuracy: 0.8772 - val_loss: 0.3396 - lr: 0.0
Epoch 13/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8937 - loss: 0.2987 - val_accuracy: 0.8768 - val_loss: 0.3380 - lr: 0.0
Epoch 14/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8945 - loss: 0.2966 - val_accuracy: 0.8774 - val_loss: 0.3366 - lr: 0.
Epoch 15/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8955 - loss: 0.2948 - val_accuracy: 0.8766 - val_loss: 0.3352 - lr: 0.0
Epoch 16/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8961 - loss: 0.2931 - val_accuracy: 0.8778 - val_loss: 0.3340 - lr: 0.0
Epoch 17/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8965 - loss: 0.2917 - val_accuracy: 0.8786 - val_loss: 0.3329 - lr: 0.0
Epoch 18/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8967 - loss: 0.2904 - val_accuracy: 0.8798 - val_loss: 0.3319 - lr: 0.0
Epoch 19/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8971 - loss: 0.2892 - val_accuracy: 0.8804 - val_loss: 0.3310 - lr: 0.0
Epoch 20/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8975 - loss: 0.2882 - val_accuracy: 0.8804 - val_loss: 0.3302 - lr: 1.0
```

Lập lịch Hằng số Từng phần (Piecewise Constant Scheduling)

```
# lr_schedule = tf.keras.optimizers.schedules.PiecewiseConstantDecay(
#     boundaries=[50_000, 80_000],
#     values=[0.01, 0.005, 0.001]
# )
# optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
import tensorflow as tf
from tensorflow.keras import layers, models

def build_and_train_model(optimizer, n_epochs=20, batch_size=32):
    # 1. Xây dựng model
    model = models.Sequential([
        layers.Flatten(input_shape=[28, 28]),
        layers.Dense(100, activation="relu"),
        layers.Dense(100, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])

    # 2. Compile với optimizer truyền vào
    model.compile(
        loss="sparse_categorical_crossentropy",
        optimizer=optimizer,
        metrics=["accuracy"]
    )

    # 3. Huấn luyện model
    history = model.fit(
        X_train, y_train,
        epochs=n_epochs,
        validation_data=(X_valid, y_valid),
        batch_size=batch_size
    )

    return history
```

```
#sử dụng
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
history_piecewise_scheduling = build_and_train_model(optimizer)
```

```
Epoch 1/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.7446 - loss: 0.7725 - val_accuracy: 0.8178 - val_loss: 0.5299
Epoch 2/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8248 - loss: 0.5040 - val_accuracy: 0.8330 - val_loss: 0.4679
Epoch 3/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8406 - loss: 0.4550 - val_accuracy: 0.8398 - val_loss: 0.4408
Epoch 4/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8502 - loss: 0.4274 - val_accuracy: 0.8438 - val_loss: 0.4243
Epoch 5/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8569 - loss: 0.4079 - val_accuracy: 0.8490 - val_loss: 0.4136
Epoch 6/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8614 - loss: 0.3926 - val_accuracy: 0.8520 - val_loss: 0.4047
Epoch 7/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8659 - loss: 0.3799 - val_accuracy: 0.8544 - val_loss: 0.3964
Epoch 8/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8688 - loss: 0.3691 - val_accuracy: 0.8590 - val_loss: 0.3903
```

```
Epoch 9/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8720 - loss: 0.3595 - val_accuracy: 0.8586 - val_loss: 0.3858
Epoch 10/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8741 - loss: 0.3509 - val_accuracy: 0.8616 - val_loss: 0.3797
Epoch 11/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8765 - loss: 0.3432 - val_accuracy: 0.8626 - val_loss: 0.3754
Epoch 12/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8786 - loss: 0.3360 - val_accuracy: 0.8626 - val_loss: 0.3725
Epoch 13/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8805 - loss: 0.3294 - val_accuracy: 0.8652 - val_loss: 0.3693
Epoch 14/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8827 - loss: 0.3232 - val_accuracy: 0.8648 - val_loss: 0.3669
Epoch 15/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8850 - loss: 0.3173 - val_accuracy: 0.8636 - val_loss: 0.3645
Epoch 16/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8869 - loss: 0.3116 - val_accuracy: 0.8650 - val_loss: 0.3622
Epoch 17/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8886 - loss: 0.3063 - val_accuracy: 0.8662 - val_loss: 0.3600
Epoch 18/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8909 - loss: 0.3013 - val_accuracy: 0.8676 - val_loss: 0.3579
Epoch 19/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8924 - loss: 0.2964 - val_accuracy: 0.8686 - val_loss: 0.3563
Epoch 20/20
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8941 - loss: 0.2919 - val_accuracy: 0.8696 - val_loss: 0.3539
```

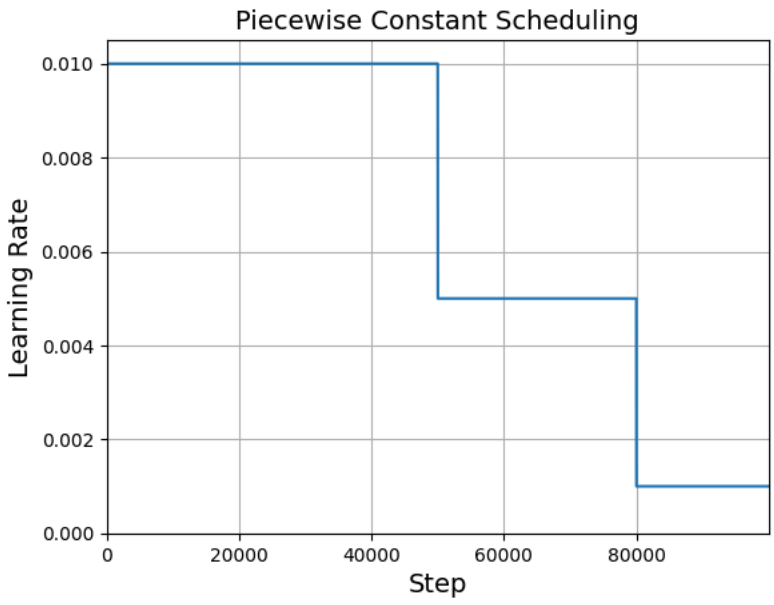
```
# extra code – this cell plots piecewise constant scheduling
import numpy as np
import matplotlib.pyplot as plt

boundaries = [50_000, 80_000]
values = [0.01, 0.005, 0.001]

steps = np.arange(100_000)

lrs = np.full(len(steps), values[0])
for boundary, value in zip(boundaries, values[1:]):
    lrs[boundary:] = value

plt.plot(steps, lrs, "-")
plt.axis([0, steps.max(), 0, 0.0105])
plt.xlabel("Step")
plt.ylabel("Learning Rate")
plt.title("Piecewise Constant Scheduling", fontsize=14)
plt.grid(True)
plt.show()
```



Giống như cách chúng ta đã làm với lập lịch theo hàm số mũ, chúng ta cũng có thể triển khai **lập lịch hằng số từng phần** theo cách thủ công:

```
def piecewise_constant_fn(epoch):
    if epoch < 5:
```

```
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001
```

```
# extra code – this cell demonstrates a more general way to define
#              piecewise constant scheduling.

def piecewise_constant(boundaries, values):
    boundaries = np.array([0] + boundaries)
    values = np.array(values)
    def piecewise_constant_fn(epoch):
        return values[(boundaries > epoch).argmax() - 1]
    return piecewise_constant_fn

piecewise_constant_fn = piecewise_constant([5, 15], [0.01, 0.005, 0.001])
```

định nghĩa đơn giản cho build_model()

```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_model():
    model = models.Sequential([
        layers.Flatten(input_shape=[28, 28]),
        layers.Dense(100, activation="relu"),
        layers.Dense(100, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    return model
```

Sử dụng

```
# Hàm điều chỉnh LR theo epoch
def piecewise_constant_fn(epoch):
    if epoch < 5:
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001

n_epochs = 25
lr0 = 0.01  # learning rate khởi đầu

# Callback scheduler
lr_scheduler = tf.keras.callbacks.LearningRateScheduler(piecewise_constant_fn)

# Xây dựng và train model
model = build_model()
optimizer = tf.keras.optimizers.Nadam(learning_rate=lr0)
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=optimizer,
    metrics=["accuracy"]
)

history = model.fit(
    X_train, y_train,
    epochs=n_epochs,
    validation_data=(X_valid, y_valid),
    callbacks=[lr_scheduler]
)
```

```
Epoch 1/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8168 - loss: 0.5134 - val_accuracy: 0.8260 - val_loss: 0.4505 - learni
Epoch 2/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8466 - loss: 0.4274 - val_accuracy: 0.8492 - val_loss: 0.4201 - learnin
Epoch 3/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8552 - loss: 0.4060 - val_accuracy: 0.8466 - val_loss: 0.4322 - learnin
Epoch 4/25
```

```
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8616 - loss: 0.3920 - val_accuracy: 0.8452 - val_loss: 0.4250 - learnin
Epoch 5/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8643 - loss: 0.3833 - val_accuracy: 0.8558 - val_loss: 0.4097 - learnin
Epoch 6/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8802 - loss: 0.3362 - val_accuracy: 0.8454 - val_loss: 0.4897 - learnin
Epoch 7/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8815 - loss: 0.3258 - val_accuracy: 0.8582 - val_loss: 0.4234 - learnin
Epoch 8/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8841 - loss: 0.3182 - val_accuracy: 0.8582 - val_loss: 0.4140 - learnin
Epoch 9/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8865 - loss: 0.3133 - val_accuracy: 0.8592 - val_loss: 0.4278 - learnin
Epoch 10/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8883 - loss: 0.3089 - val_accuracy: 0.8500 - val_loss: 0.4645 - learnin
Epoch 11/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8881 - loss: 0.3059 - val_accuracy: 0.8604 - val_loss: 0.4608 - learnin
Epoch 12/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8890 - loss: 0.3057 - val_accuracy: 0.8606 - val_loss: 0.4718 - learnin
Epoch 13/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8915 - loss: 0.2992 - val_accuracy: 0.8574 - val_loss: 0.4582 - learnin
Epoch 14/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8911 - loss: 0.2993 - val_accuracy: 0.8600 - val_loss: 0.4442 - learnin
Epoch 15/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8923 - loss: 0.2935 - val_accuracy: 0.8662 - val_loss: 0.4392 - learnin
Epoch 16/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9029 - loss: 0.2633 - val_accuracy: 0.8758 - val_loss: 0.4096 - learni
Epoch 17/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9054 - loss: 0.2552 - val_accuracy: 0.8752 - val_loss: 0.4087 - learnin
Epoch 18/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9066 - loss: 0.2518 - val_accuracy: 0.8744 - val_loss: 0.4196 - learnin
Epoch 19/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9080 - loss: 0.2487 - val_accuracy: 0.8756 - val_loss: 0.4259 - learnin
Epoch 20/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9086 - loss: 0.2463 - val_accuracy: 0.8748 - val_loss: 0.4276 - learnin
Epoch 21/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9093 - loss: 0.2438 - val_accuracy: 0.8762 - val_loss: 0.4384 - learnin
Epoch 22/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9097 - loss: 0.2420 - val_accuracy: 0.8756 - val_loss: 0.4481 - learnin
Epoch 23/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9106 - loss: 0.2400 - val_accuracy: 0.8748 - val_loss: 0.4455 - learnin
Epoch 24/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9110 - loss: 0.2383 - val_accuracy: 0.8750 - val_loss: 0.4599 - learnin
Epoch 25/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9115 - loss: 0.2370 - val_accuracy: 0.8748 - val_loss: 0.4662 - learnin
```

Chúng ta đã xem xét `InverseTimeDecay`, `ExponentialDecay` và `PiecewiseConstantDecay`. Một vài bộ lập lịch khác có sẵn trong `tf.keras.optimizers.schedules`, đây là danh sách đầy đủ:

```
for name in sorted(dir(tf.keras.optimizers.schedules)):
    if name[0] == name[0].lower():  # must start with capital letter
        continue
    scheduler_class = getattr(tf.keras.optimizers.schedules, name)
    print(f"• {name} – {scheduler_class.__doc__.splitlines()[0]}")
```

```
• CosineDecay – A `LearningRateSchedule` that uses a cosine decay with optional warmup.
• CosineDecayRestarts – A `LearningRateSchedule` that uses a cosine decay schedule with restarts.
• ExponentialDecay – A `LearningRateSchedule` that uses an exponential decay schedule.
• InverseTimeDecay – A `LearningRateSchedule` that uses an inverse time decay schedule.
• LearningRateSchedule – The learning rate schedule base class.
• PiecewiseConstantDecay – A `LearningRateSchedule` that uses a piecewise constant decay schedule.
• PolynomialDecay – A `LearningRateSchedule` that uses a polynomial decay schedule.
```

Lập lịch Theo Hiệu suất (Performance Scheduling)

```
# extra code – build and compile the model

model = build_model()
optimizer = tf.keras.optimizers.SGD(learning_rate=lr0)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
```

```
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)
history = model.fit(X_train, y_train, epochs=n_epochs,
                    validation_data=(X_valid, y_valid),
                    callbacks=[lr_scheduler])
```

```
Epoch 1/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.7390 - loss: 0.7843 - val_accuracy: 0.8190 - val_loss: 0.5246 - learnin
```

```
Epoch 2/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8239 - loss: 0.5066 - val_accuracy: 0.8294 - val_loss: 0.4674 - learnin
Epoch 3/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8406 - loss: 0.4582 - val_accuracy: 0.8396 - val_loss: 0.4438 - learnin
Epoch 4/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8488 - loss: 0.4300 - val_accuracy: 0.8472 - val_loss: 0.4257 - learnin
Epoch 5/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8553 - loss: 0.4100 - val_accuracy: 0.8518 - val_loss: 0.4147 - learnin
Epoch 6/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8600 - loss: 0.3943 - val_accuracy: 0.8538 - val_loss: 0.4039 - learnin
Epoch 7/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8648 - loss: 0.3814 - val_accuracy: 0.8578 - val_loss: 0.3951 - learnin
Epoch 8/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8689 - loss: 0.3701 - val_accuracy: 0.8624 - val_loss: 0.3864 - learnin
Epoch 9/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8725 - loss: 0.3598 - val_accuracy: 0.8632 - val_loss: 0.3792 - learnin
Epoch 10/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8753 - loss: 0.3506 - val_accuracy: 0.8638 - val_loss: 0.3754 - learnin
Epoch 11/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8779 - loss: 0.3422 - val_accuracy: 0.8646 - val_loss: 0.3709 - learnin
Epoch 12/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8806 - loss: 0.3345 - val_accuracy: 0.8652 - val_loss: 0.3678 - learnin
Epoch 13/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8828 - loss: 0.3273 - val_accuracy: 0.8678 - val_loss: 0.3641 - learnin
Epoch 14/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8848 - loss: 0.3208 - val_accuracy: 0.8696 - val_loss: 0.3603 - learni
Epoch 15/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8867 - loss: 0.3146 - val_accuracy: 0.8690 - val_loss: 0.3576 - learnin
Epoch 16/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8886 - loss: 0.3089 - val_accuracy: 0.8692 - val_loss: 0.3541 - learnin
Epoch 17/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8905 - loss: 0.3036 - val_accuracy: 0.8702 - val_loss: 0.3533 - learnin
Epoch 18/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8918 - loss: 0.2987 - val_accuracy: 0.8712 - val_loss: 0.3510 - learnin
Epoch 19/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8937 - loss: 0.2939 - val_accuracy: 0.8724 - val_loss: 0.3482 - learnin
Epoch 20/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8956 - loss: 0.2893 - val_accuracy: 0.8732 - val_loss: 0.3468 - learnin
Epoch 21/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8970 - loss: 0.2850 - val_accuracy: 0.8738 - val_loss: 0.3452 - learnin
Epoch 22/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8985 - loss: 0.2809 - val_accuracy: 0.8742 - val_loss: 0.3433 - learnin
Epoch 23/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8999 - loss: 0.2769 - val_accuracy: 0.8762 - val_loss: 0.3424 - learnin
Epoch 24/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9016 - loss: 0.2731 - val_accuracy: 0.8744 - val_loss: 0.3426 - learnin
Epoch 25/25
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9026 - loss: 0.2694 - val_accuracy: 0.8750 - val_loss: 0.3421 - learnin
```

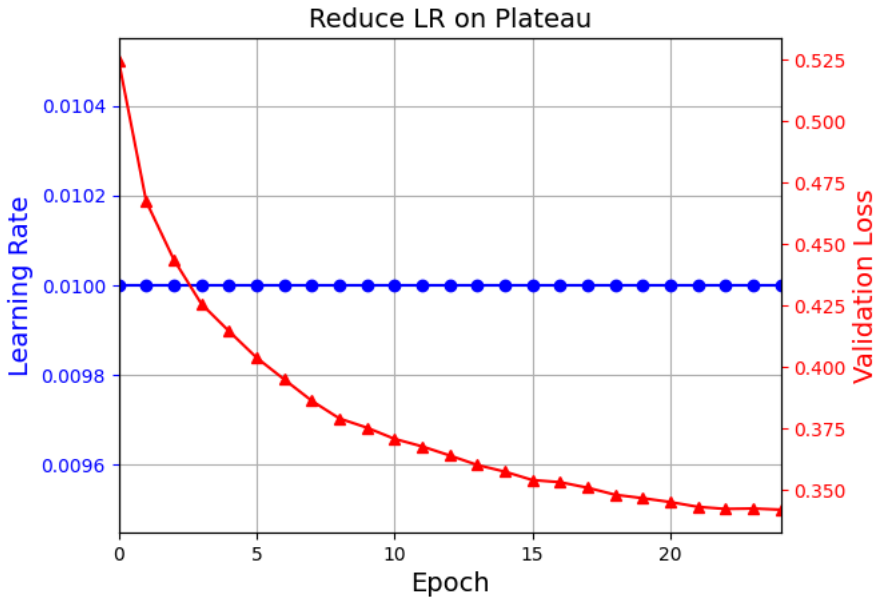
```
# extra code – this cell plots performance scheduling

#plt.plot(history.epoch, history.history["lr"], "bo-")
plt.plot(history.epoch, history.history["learning_rate"], "bo-")

plt.xlabel("Epoch")
plt.ylabel("Learning Rate", color='b')
plt.tick_params('y', colors='b')
plt.gca().set_xlim(0, n_epochs - 1)
plt.grid(True)

ax2 = plt.gca().twinx()
ax2.plot(history.epoch, history.history["val_loss"], "r^-")
ax2.set_ylabel('Validation Loss', color='r')
ax2.tick_params('y', colors='r')

plt.title("Reduce LR on Plateau", fontsize=14)
plt.show()
```



Lập lịch 1 Chu kỳ (1Cycle scheduling)

Callback tùy chỉnh `ExponentialLearningRate` cập nhật **tốc độ học** trong quá trình huấn luyện, ở cuối **mỗi batch**. Nó nhân tốc độ học với một hằng số `factor`. Nó cũng lưu lại tốc độ học và độ lỗi ở mỗi batch. Vì `logs["loss"]` thực chất là **độ lỗi trung bình** kể từ đầu **epoch**, và chúng ta muốn lưu độ lỗi của **batch** hiện tại, nên chúng ta phải tính **giá trị trung bình nhân với số lượng batch** kể từ đầu epoch để có được tổng độ lỗi cho đến nay, sau đó chúng ta trừ đi tổng độ lỗi ở batch trước để có được độ lỗi của batch hiện tại.

```
K = tf.keras.backend

class ExponentialLearningRate(tf.keras.callbacks.Callback):
    def __init__(self, factor):
        self.factor = factor
        self.rates = []
        self.losses = []

    def on_epoch_begin(self, epoch, logs=None):
        self.sum_of_epoch_losses = 0

    def on_batch_end(self, batch, logs=None):
        mean_epoch_loss = logs["loss"]  # the epoch's mean loss so far
        new_sum_of_epoch_losses = mean_epoch_loss * (batch + 1)
        batch_loss = new_sum_of_epoch_losses - self.sum_of_epoch_losses
        self.sum_of_epoch_losses = new_sum_of_epoch_losses
        self.rates.append(K.get_value(self.model.optimizer.learning_rate))
        self.losses.append(batch_loss)
        K.set_value(self.model.optimizer.learning_rate,
                    self.model.optimizer.learning_rate * self.factor)
```

Hàm `find_learning_rate()` huấn luyện mô hình bằng cách sử dụng callback `ExponentialLearningRate`, và nó trả về **tốc độ học** và **độ lỗi batch** tương ứng. Cuối cùng, nó **khôi phục (restores)** mô hình và **bộ tối ưu hóa (optimizer)** của nó về trạng thái ban đầu.

```
def find_learning_rate(model, X, y, epochs=1, batch_size=32, min_rate=1e-4,
                       max_rate=1):
    init_weights = model.get_weights()
    iterations = math.ceil(len(X) / batch_size) * epochs
    factor = (max_rate / min_rate) ** (1 / iterations)
    init_lr = K.get_value(model.optimizer.learning_rate)
    K.set_value(model.optimizer.learning_rate, min_rate)
    exp_lr = ExponentialLearningRate(factor)
    history = model.fit(X, y, epochs=epochs, batch_size=batch_size,
                        callbacks=[exp_lr])
    K.set_value(model.optimizer.learning_rate, init_lr)
    model.set_weights(init_weights)
    return exp_lr.rates, exp_lr.losses
```

Hàm `plot_lr_vs_loss()` vẽ biểu đồ **tốc độ học** so với **độ lỗi**. **Tốc độ học tối ưu** để sử dụng làm tốc độ học tối đa trong **1 chu kỳ (1cycle)** nằm gần điểm thấp nhất của đường cong.

```
def plot_lr_vs_loss(rates, losses):
    plt.plot(rates, losses, "b")
    plt.gca().set_xscale('log')
    max_loss = losses[0] + min(losses)
    plt.hlines(min(losses), min(rates), max(rates), color="k")
    plt.axis([min(rates), max(rates), 0, max_loss])
    plt.xlabel("Learning rate")
    plt.ylabel("Loss")
    plt.grid()
```

Hãy xây dựng một mô hình Fashion MNIST đơn giản và biên dịch nó:

```
# model = build_model()
# # model.compile(loss="sparse_categorical_crossentropy",
# #               optimizer=tf.keras.optimizers.SGD(learning_rate=0.001),
# #               metrics=["accuracy"])
import tensorflow as tf
from tensorflow.keras import layers, models

# Tạo mô hình đơn giản (ví dụ cho Fashion MNIST)
def build_model():
    model = models.Sequential([
        layers.Flatten(input_shape=[28, 28]),
        layers.Dense(100, activation="relu"),
        layers.Dense(100, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    return model

# Khởi tạo optimizer đúng chuẩn
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)

# Biên dịch mô hình
model = build_model()
model.compile(
    optimizer=optimizer,
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
# ✅ Bước 2: nếu find_learning_rate vẫn dùng K.set_value thì ép kiểu
import numpy as np

def lr_finder(model, X, y, epochs=1, batch_size=128, min_lr=1e-5, max_lr=10):
    num_batches = len(X) // batch_size * epochs
    lr_mult = (max_lr / min_lr) ** (1 / num_batches)
    lrs = []
    losses = []

    # Đặt LR khởi đầu
    model.optimizer.learning_rate.assign(min_lr)

    for epoch in range(epochs):
        for i in range(0, len(X), batch_size):
            X_batch = X[i:i+batch_size]
            y_batch = y[i:i+batch_size]

            logs = model.train_on_batch(X_batch, y_batch)
            loss = logs if np.isscalar(logs) else logs[0]
            losses.append(loss)
            lrs.append(float(model.optimizer.learning_rate.numpy()))

            # tăng LR theo cấp số nhân
            model.optimizer.learning_rate.assign(model.optimizer.learning_rate * lr_mult)

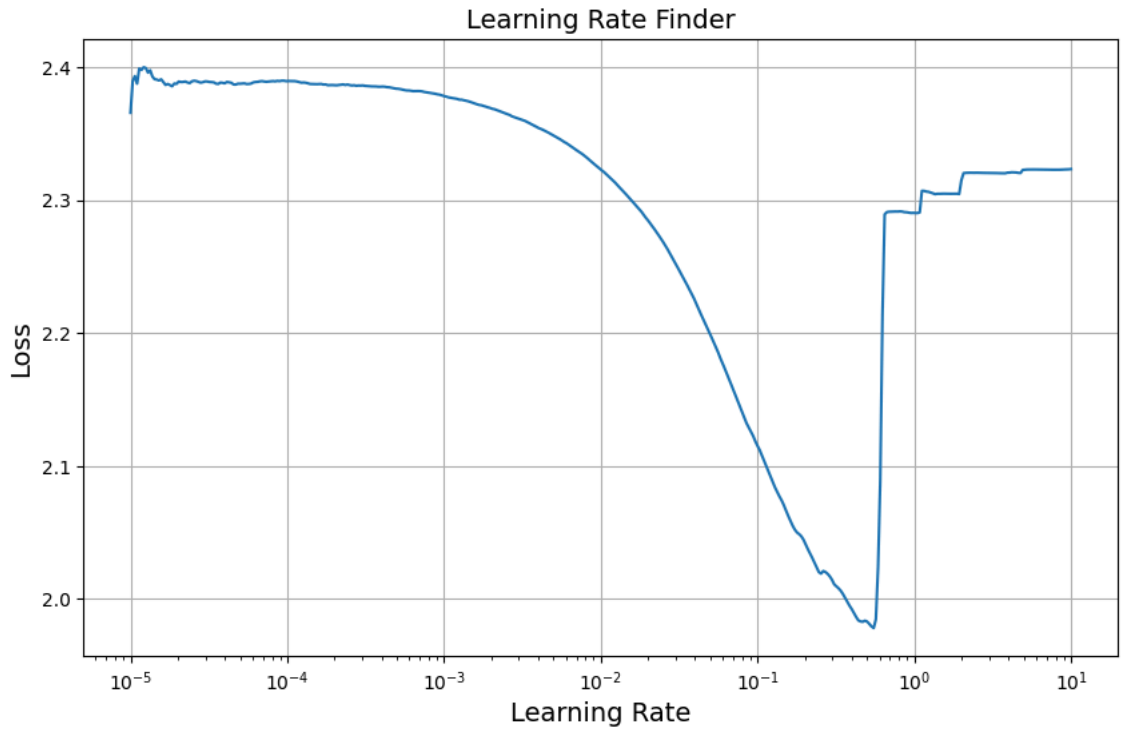
    return np.array(lrs), np.array(losses)
```

Bây giờ hãy tìm **tốc độ học tối đa tối ưu** cho **1 chu kỳ (1cycle)**:

```
# batch_size = 128
# rates, losses = find_learning_rate(model, X_train, y_train, epochs=1,
#                                    batch_size=batch_size)
# plot_lr_vs_loss(rates, losses)
# ✅ Bước 3: chạy LR finder
import matplotlib.pyplot as plt

# Chạy LR Finder
batch_size = 128
rates, losses = lr_finder(model, X_train, y_train, epochs=1, batch_size=batch_size)

# Vẽ Learning Rate vs Loss
plt.figure(figsize=(10,6))
plt.plot(rates, losses)
plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Loss")
plt.title("Learning Rate Finder")
plt.grid(True)
plt.show()
```



Có vẻ như tốc độ học tối đa để sử dụng cho **1 chu kỳ (1cycle)** là khoảng 10^{-1}.

Callback tùy chỉnh `OneCycleScheduler` cập nhật **tốc độ học** ở đầu **mỗi batch**. Nó áp dụng logic được mô tả trong sách: tăng tốc độ học tuyến tính trong khoảng một nửa thời gian huấn luyện, sau đó giảm nó tuyến tính trở lại tốc độ học ban đầu, và cuối cùng giảm nó xuống gần bằng không một cách tuyến tính trong phần rất cuối của quá trình huấn luyện.

```
import math
import tensorflow as tf

class OneCycleScheduler(tf.keras.callbacks.Callback):
    def __init__(self, total_steps, max_lr=0.1, start_lr=None, last_lr=None):
        super().__init__()
        self.total_steps = total_steps
        self.max_lr = max_lr
        self.start_lr = start_lr or max_lr / 10
        self.last_lr = last_lr or self.start_lr / 1e4
        self.iteration = 0
```

```
    def lr_schedule(self, t):
        if t <= 0.3:
            return self.start_lr + t / 0.3 * (self.max_lr - self.start_lr)
        elif t <= 0.7:
            return self.max_lr - (t - 0.3) / 0.4 * (self.max_lr - self.start_lr)
        else:
            return self.start_lr - (t - 0.7) / 0.3 * (self.start_lr - self.last_lr)

    def on_batch_begin(self, batch, logs=None):
        t = self.iteration / self.total_steps
        lr = self.lr_schedule(t)
        # ✅ Sửa ở đây
        self.model.optimizer.learning_rate.assign(lr)
        self.iteration += 1

    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        logs['learning_rate'] = float(self.model.optimizer.learning_rate.numpy())
```

Hãy xây dựng và biên dịch một mô hình Fashion MNIST đơn giản, sau đó huấn luyện nó bằng cách sử dụng callback `OneCycleScheduler`:

```
import math

model = build_model()
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=tf.keras.optimizers.SGD(),
    metrics=["accuracy"]
)

n_epochs = 25
total_steps = math.ceil(len(X_train) / batch_size) * n_epochs
onecycle = OneCycleScheduler(total_steps, max_lr=0.1)

history = model.fit(
    X_train, y_train,
    epochs=n_epochs,
    batch_size=batch_size,
    validation_data=(X_valid, y_valid),
    callbacks=[onecycle]
)
```

```
Epoch 1/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.6507 - loss: 1.1336 - val_accuracy: 0.7730 - val_loss: 0.6773 - learning_
Epoch 2/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.7884 - loss: 0.6175 - val_accuracy: 0.8108 - val_loss: 0.5428 - learning_
Epoch 3/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8167 - loss: 0.5299 - val_accuracy: 0.8252 - val_loss: 0.5013 - learning_
Epoch 4/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8323 - loss: 0.4782 - val_accuracy: 0.8234 - val_loss: 0.4906 - learning_
Epoch 5/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8428 - loss: 0.4425 - val_accuracy: 0.8252 - val_loss: 0.4832 - learning_
Epoch 6/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8515 - loss: 0.4157 - val_accuracy: 0.8318 - val_loss: 0.4661 - learning_
Epoch 7/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.8583 - loss: 0.3948 - val_accuracy: 0.8392 - val_loss: 0.4449 - learning_
Epoch 8/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8647 - loss: 0.3758 - val_accuracy: 0.8396 - val_loss: 0.4315 - learning_
Epoch 9/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8723 - loss: 0.3517 - val_accuracy: 0.8458 - val_loss: 0.4115 - learning_
Epoch 10/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8788 - loss: 0.3332 - val_accuracy: 0.8534 - val_loss: 0.3946 - learning_
Epoch 11/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8841 - loss: 0.3183 - val_accuracy: 0.8582 - val_loss: 0.3817 - learning_
Epoch 12/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.8885 - loss: 0.3064 - val_accuracy: 0.8620 - val_loss: 0.3731 - learning_
Epoch 13/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8926 - loss: 0.2962 - val_accuracy: 0.8636 - val_loss: 0.3608 - learning_
Epoch 14/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8964 - loss: 0.2877 - val_accuracy: 0.8678 - val_loss: 0.3527 - learning_
Epoch 15/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8990 - loss: 0.2804 - val_accuracy: 0.8726 - val_loss: 0.3446 - learning_
Epoch 16/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.9009 - loss: 0.2742 - val_accuracy: 0.8770 - val_loss: 0.3382 - learning_
Epoch 17/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9031 - loss: 0.2691 - val_accuracy: 0.8816 - val_loss: 0.3305 - learning_
```

```
Epoch 18/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9043 - loss: 0.2652 - val_accuracy: 0.8816 - val_loss: 0.3258 - learning_
Epoch 19/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9048 - loss: 0.2633 - val_accuracy: 0.8808 - val_loss: 0.3244 - learning_
Epoch 20/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9052 - loss: 0.2620 - val_accuracy: 0.8810 - val_loss: 0.3230 - learning_
Epoch 21/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9057 - loss: 0.2609 - val_accuracy: 0.8804 - val_loss: 0.3216 - learning_
Epoch 22/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9060 - loss: 0.2599 - val_accuracy: 0.8806 - val_loss: 0.3205 - learning_
Epoch 23/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9063 - loss: 0.2591 - val_accuracy: 0.8810 - val_loss: 0.3196 - learning_
Epoch 24/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9067 - loss: 0.2584 - val_accuracy: 0.8822 - val_loss: 0.3189 - learning_
Epoch 25/25
430/430 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9067 - loss: 0.2578 - val_accuracy: 0.8818 - val_loss: 0.3184 - learning_
```

Tránh Hiện tượng Quá khớp Thông qua Điều chuẩn hóa (Regularization)

Điều chuẩn hóa ℓ_1 và ℓ_2

```
layer = tf.keras.layers.Dense(100, activation="relu",
                              kernel_initializer="he_normal",
                              kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

Hoặc sử dụng `l1(0.1)` cho **điều chuẩn hóa** ℓ_1 với hệ số 0.1, hoặc `l1_l2(0.1, 0.01)` cho cả điều chuẩn hóa ℓ_1 và ℓ_2, với các hệ số lần lượt là 0.1 và 0.01.

```
tf.random.set_seed(42)  # extra code – for reproducibility
```

```
from functools import partial

RegularizedDense = partial(tf.keras.layers.Dense,
                           activation="relu",
                           kernel_initializer="he_normal",
                           kernel_regularizer=tf.keras.regularizers.l2(0.01))

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    RegularizedDense(100),
    RegularizedDense(100),
    RegularizedDense(10, activation="softmax")
])
```

```
# extra code – compile and train the model
optimizer = tf.keras.optimizers.SGD(learning_rate=0.02)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=2,
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.7677 - loss: 3.1508 - val_accuracy: 0.8176 - val_loss: 1.8763
Epoch 2/2
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8110 - loss: 1.4336 - val_accuracy: 0.8218 - val_loss: 1.1236
```

Dropout

```
tf.random.set_seed(42)  # extra code – for reproducibility
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
```

```
                        kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

```
# extra code – compile and train the model
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.7523 - loss: 0.6776 - val_accuracy: 0.8362 - val_loss: 0.4587
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8117 - loss: 0.5151 - val_accuracy: 0.8518 - val_loss: 0.4045
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8259 - loss: 0.4762 - val_accuracy: 0.8554 - val_loss: 0.3970
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8330 - loss: 0.4551 - val_accuracy: 0.8612 - val_loss: 0.3720
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8383 - loss: 0.4397 - val_accuracy: 0.8554 - val_loss: 0.3783
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8425 - loss: 0.4267 - val_accuracy: 0.8696 - val_loss: 0.3616
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8448 - loss: 0.4147 - val_accuracy: 0.8692 - val_loss: 0.3666
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8496 - loss: 0.4080 - val_accuracy: 0.8658 - val_loss: 0.3590
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8511 - loss: 0.4019 - val_accuracy: 0.8686 - val_loss: 0.3541
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8529 - loss: 0.3947 - val_accuracy: 0.8706 - val_loss: 0.3529
```

Độ chính xác khi **huấn luyện (training accuracy)** có vẻ như thấp hơn độ chính xác khi **xác thực (validation accuracy)**, nhưng điều này chỉ là vì **dropout** chỉ hoạt động trong quá trình huấn luyện. Nếu chúng ta **đánh giá (evaluate)** mô hình trên tập huấn luyện sau khi huấn luyện (tức là với dropout đã **tắt**), chúng ta sẽ có được độ chính xác huấn luyện "thực sự", hơi cao hơn một chút so với độ chính xác xác thực và độ chính xác kiểm tra:

```
model.evaluate(X_train, y_train)
```

```
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8827 - loss: 0.3124
[0.3124453127384186, 0.8827454447746277]
```

```
model.evaluate(X_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8621 - loss: 0.3700
[0.36998388171195984, 0.8621000051498413]
```

Lưu ý: hãy đảm bảo sử dụng `AlphaDropout` thay vì `Dropout` nếu bạn muốn xây dựng một **mạng nơ-ron tự chuẩn hóa (self-normalizing neural net)** sử dụng **SELU**.

MC Dropout

```
tf.random.set_seed(42)  # extra code – for reproducibility
```

```
y_probas = np.stack([model(X_test, training=True)
                     for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```

```
model.predict(X_test[:1]).round(3)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step
array([[0.   , 0.   , 0.   , 0.   , 0.   , 0.033, 0.   , 0.167, 0.   ,
        0.8  ]], dtype=float32)
```

```
y_proba[0].round(3)
```

```
array([0.   , 0.   , 0.   , 0.   , 0.   , 0.073, 0.   , 0.244, 0.001,
       0.682], dtype=float32)
```

```
y_std = y_probas.std(axis=0)
y_std[0].round(3)
```

```
array([0.   , 0.   , 0.   , 0.001, 0.   , 0.106, 0.   , 0.226, 0.004,
       0.228], dtype=float32)
```

```
y_pred = y_proba.argmax(axis=1)
accuracy = (y_pred == y_test).sum() / len(y_test)
accuracy
```

```
np.float64(0.8633)
```

```
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=None):
        return super().call(inputs, training=True)
```

```
# extra code – shows how to convert Dropout to MCDropout in a Sequential model
Dropout = tf.keras.layers.Dropout
mc_model = tf.keras.Sequential([
    MCDropout(layer.rate) if isinstance(layer, Dropout) else layer
    for layer in model.layers
])
mc_model.set_weights(model.get_weights())
```

```
mc_model.summary()
```

Model: "sequential_25"

Layer (type)	Output Shape	Param #
flatten_22 (Flatten)	(None, 784)	0
mc_dropout (MCDropout)	(None, 784)	0
dense_83 (Dense)	(None, 100)	78,500
mc_dropout_1 (MCDropout)	(None, 100)	0
dense_84 (Dense)	(None, 100)	10,100
mc_dropout_2 (MCDropout)	(None, 100)	0
dense_85 (Dense)	(None, 10)	1,010

Total params: 89,610 (350.04 KB)
Trainable params: 89,610 (350.04 KB)
Non-trainable params: 0 (0.00 B)

Bây giờ chúng ta có thể sử dụng mô hình với **MC Dropout**:

```
# extra code – shows that the model works without retraining
tf.random.set_seed(42)
np.mean([mc_model.predict(X_test[:1])
         for sample in range(100)], axis=0).round(2)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
array([[0. , 0. , 0. , 0. , 0. , 0.1, 0. , 0.2, 0. , 0.7]], dtype=float32)
```

Chuẩn tối đa (Max norm)

```
dense = tf.keras.layers.Dense(
    100, activation="relu", kernel_initializer="he_normal",
    kernel_constraint=tf.keras.constraints.max_norm(1.))
```

```
# extra code – shows how to apply max norm to every hidden layer in a model

MaxNormDense = partial(tf.keras.layers.Dense,
                       activation="relu", kernel_initializer="he_normal",
                       kernel_constraint=tf.keras.constraints.max_norm(1.))

tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    MaxNormDense(100),
    MaxNormDense(100),
    tf.keras.layers.Dense(10, activation="softmax")
])
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8049 - loss: 0.5443 - val_accuracy: 0.8384 - val_loss: 0.4269
Epoch 2/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8496 - loss: 0.4092 - val_accuracy: 0.8494 - val_loss: 0.3947
Epoch 3/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8623 - loss: 0.3753 - val_accuracy: 0.8396 - val_loss: 0.3979
Epoch 4/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8680 - loss: 0.3556 - val_accuracy: 0.8456 - val_loss: 0.3901
Epoch 5/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8728 - loss: 0.3420 - val_accuracy: 0.8396 - val_loss: 0.3989
Epoch 6/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8750 - loss: 0.3354 - val_accuracy: 0.8498 - val_loss: 0.3800
Epoch 7/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8781 - loss: 0.3289 - val_accuracy: 0.8526 - val_loss: 0.3817
Epoch 8/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8801 - loss: 0.3231 - val_accuracy: 0.8396 - val_loss: 0.3940
Epoch 9/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8808 - loss: 0.3190 - val_accuracy: 0.8380 - val_loss: 0.4094
```

```
Epoch 10/10
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8811 - loss: 0.3173 - val_accuracy: 0.8450 - val_loss: 0.3980
```

Luyện tập

1. tới 7.

1. **Khởi tạo Glorot** và **Khởi tạo He** được thiết kế để làm cho độ lệch chuẩn đầu ra càng gần với độ lệch chuẩn đầu vào càng tốt, ít nhất là ở giai đoạn bắt đầu huấn luyện. Điều này làm giảm **vấn đề triệt tiêu/bùng nổ độ dốc**.
2. Không, tất cả các trọng số nên được **lấy mẫu một cách độc lập**; chúng không nên có cùng một giá trị khởi tạo. Một mục tiêu quan trọng của việc lấy mẫu trọng số ngẫu nhiên là để **phá vỡ tính đối xứng (break symmetry)**: nếu tất cả các trọng số có cùng một giá trị ban đầu, ngay cả khi giá trị đó không phải là zero, thì tính đối xứng sẽ không bị phá vỡ (tức là tất cả các nơ-ron trong một lớp nhất định đều tương đương), và **lan truyền ngược (backpropagation)** sẽ không thể phá vỡ nó. Cụ thể, điều này có nghĩa là tất cả các nơ-ron trong bất kỳ lớp nào cũng sẽ luôn có cùng một trọng số. Nó giống như chỉ có một nơ-ron trên mỗi lớp, và chậm hơn nhiều. Hầu như không thể để một cấu hình như vậy hội tụ về một giải pháp tốt.
3. Việc khởi tạo các **hệ số thiên vị (bias terms)** bằng zero là hoàn toàn ổn. Một số người thích khởi tạo chúng giống như trọng số, và điều đó cũng được; nó không tạo ra nhiều khác biệt.
4. **ReLU** thường là một lựa chọn mặc định tốt cho các lớp ẩn, vì nó nhanh và mang lại kết quả tốt. Khả năng xuất ra chính xác giá trị zero của nó cũng có thể hữu ích trong một số trường hợp (ví dụ, xem Chương 17). Hơn nữa, đôi khi nó có thể hưởng lợi từ các triển khai được tối ưu hóa cũng như từ sự tăng tốc phần cứng. Các biến thể **leaky ReLU** của ReLU có thể cải thiện chất lượng của mô hình mà không làm giảm tốc độ quá nhiều so với ReLU. Đối với các mạng nơ-ron lớn và các vấn đề phức tạp hơn, **GLU, Swish** và **Mish** có thể mang lại cho bạn một mô hình chất lượng cao hơn một chút, nhưng chúng có **chi phí tính toán (computational cost)**. **Hàm tang hyperbolic (tanh)** có thể hữu ích ở lớp đầu ra nếu bạn cần xuất ra một số trong một phạm vi cố định (mặc định là từ –1 đến 1), nhưng ngày nay nó không được sử dụng nhiều trong các lớp ẩn, ngoại trừ trong các mạng lặp (recurrent nets). **Hàm kích hoạt sigmoid** cũng hữu ích ở lớp đầu ra khi bạn cần ước tính một **xác suất** (ví dụ, cho phân loại nhị phân), nhưng nó hiếm khi được sử dụng trong các lớp ẩn (có những ngoại lệ—ví dụ, cho lớp mã hóa của các bộ **tự mã hóa biến phân (variational autoencoders)**; xem Chương 17). **Hàm kích hoạt softplus** hữu ích ở lớp đầu ra khi bạn cần đảm bảo rằng đầu ra sẽ luôn dương. **Hàm kích hoạt softmax** hữu ích ở lớp đầu ra để ước tính xác suất cho các lớp loại trừ lẫn nhau, nhưng nó hiếm khi (nếu có) được sử dụng trong các lớp ẩn.
5. Nếu bạn đặt **siêu tham số** `momentum` quá gần 1 (ví dụ: 0.99999) khi sử dụng trình tối ưu hóa `SGD`, thì thuật toán có thể sẽ đạt được tốc độ rất lớn, hy vọng là di chuyển gần về phía **cực tiểu toàn cục (global minimum)**, nhưng **động lượng** của nó sẽ mang nó vượt qua ngay cực tiểu đó. Sau đó, nó sẽ chậm lại và quay lại, tăng tốc trở lại, lại vượt quá mục tiêu, và cứ thế tiếp diễn. Nó có thể dao động theo cách này nhiều lần trước khi hội tụ, vì vậy nhìn chung sẽ mất nhiều thời gian hơn để hội tụ so với việc sử dụng giá trị `momentum` nhỏ hơn.
6. Một cách để tạo ra một **mô hình thưa thớt (sparse model)** (tức là với hầu hết các trọng số bằng zero) là huấn luyện mô hình bình thường, sau đó đặt các trọng số rất nhỏ bằng zero. Để có độ thưa thớt cao hơn, bạn có thể áp dụng **điều chuẩn hóa** ℓ_1 trong quá trình huấn luyện, điều này thúc đẩy trình tối ưu hóa hướng tới tính thưa thớt. Một lựa chọn thứ ba là sử dụng **TensorFlow Model Optimization Toolkit**.
7. Có, **dropout** làm chậm quá trình huấn luyện, nói chung là khoảng gấp đôi. Tuy nhiên, nó **không ảnh hưởng** đến tốc độ **suy luận (inference speed)** vì nó chỉ được bật trong quá trình huấn luyện. **MC Dropout** hoàn toàn giống như dropout trong quá trình huấn luyện, nhưng nó **vẫn hoạt động** trong quá trình suy luận, vì vậy mỗi lần suy luận bị chậm lại một chút. Quan trọng hơn, khi sử dụng MC Dropout, bạn thường muốn chạy suy luận **10 lần trở lên** để có được các dự đoán tốt hơn. Điều này có nghĩa là việc đưa ra dự đoán bị chậm lại gấp 10 lần hoặc hơn.

8. Học Sâu trên CIFAR10

a.

*Bài tập: Xây dựng một **Mạng Nơ-ron Sâu (DNN)** với 20 lớp ẩn, mỗi lớp có 100 nơ-ron (quá nhiều, nhưng đó là mục đích của bài tập này). Sử dụng **khởi tạo He** và **hàm kích hoạt Swish**.*

```
tf.random.set_seed(42)

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[32, 32, 3]))
for _ in range(20):
```

```
    model.add(tf.keras.layers.Dense(100,
                                    activation="swish",
                                    kernel_initializer="he_normal"))
```

b.

*Bài tập: Sử dụng **tối ưu hóa Nadam** và **dừng sớm (early stopping)**, huấn luyện mạng trên tập dữ liệu **CIFAR10**. Bạn có thể tải nó bằng `tf.keras.datasets.cifar10.load_data()`. Tập dữ liệu bao gồm 60.000 hình ảnh màu 32 × 32 pixel (50.000 cho huấn luyện, 10.000 cho kiểm tra) với 10 lớp, vì vậy bạn sẽ cần một **lớp đầu ra softmax** với 10 nơ-ron. Hãy nhớ tìm kiếm **tốc độ học phù hợp** mỗi khi bạn thay đổi kiến trúc hoặc siêu tham số của mô hình.*

Hãy thêm **lớp đầu ra** vào mô hình:

```
    model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Hãy sử dụng **bộ tối ưu hóa Nadam** với **tốc độ học** là $5e-5$. Tôi đã thử các tốc độ học $1e-5, 3e-5, 1e-4, 3e-4, 1e-3, 3e-3$ và $1e-2$, và tôi đã so sánh các **đường cong học tập (learning curves)** của chúng cho mỗi 10 **epoch** (sử dụng callback **TensorBoard**, ở dưới). Các tốc độ học $3e-5$ và $1e-4$ khá tốt, vì vậy tôi đã thử $5e-5$, kết quả là hơi tốt hơn một chút.

```
    optimizer = tf.keras.optimizers.Nadam(learning_rate=5e-5)
    model.compile(loss="sparse_categorical_crossentropy",
                  optimizer=optimizer,
                  metrics=["accuracy"])
```

Hãy tải tập dữ liệu **CIFAR10**. Chúng ta cũng muốn sử dụng **dừng sớm (early stopping)**, vì vậy chúng ta cần một **tập xác thực (validation set)**. Hãy sử dụng 5.000 hình ảnh đầu tiên của tập huấn luyện gốc làm tập xác thực:

```
    cifar10 = tf.keras.datasets.cifar10.load_data()
    (X_train_full, y_train_full), (X_test, y_test) = cifar10

    X_train = X_train_full[5000:]
    y_train = y_train_full[5000:]
    X_valid = X_train_full[:5000]
    y_valid = y_train_full[:5000]
```

```
    Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
    170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1252s 7us/step
```

Bây giờ chúng ta có thể tạo các **callback** cần thiết và huấn luyện mô hình:

```
    early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=20,
                                                         restore_best_weights=True)
    model_checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_cifar10_model",
                                                             save_best_only=True)
    run_index = 1 # increment every time you train the model
    run_logdir = Path() / "my_cifar10_logs" / f"run_{run_index:03d}"
    tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir)
    callbacks = [early_stopping_cb, model_checkpoint_cb, tensorboard_cb]
```

```
    ---------------------------------------------------------------------------
    ValueError                                Traceback (most recent call last)
    /tmp/ipykernel_3863/829672061.py in <cell line: 0>()
          1 early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=20,
          2                                                      restore_best_weights=True)
    ----> 3 model_checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_cifar10_model",
          4                                                          save_best_only=True)
          5 run_index = 1 # increment every time you train the model

    /usr/local/lib/python3.13/dist-packages/keras/src/callbacks/model_checkpoint.py in __init__(self, filepath, monitor, verbose,
    save_best_only, save_weights_only, mode, save_freq, initial_value_threshold)
        163                 self.filepath.endswith(ext) for ext in (".keras", ".h5")
        164             ):
    --> 165                 raise ValueError(
        166                     "The filepath provided must end in `.keras` "
        167                     "(Keras model format). Received: "

    ValueError: The filepath provided must end in `.keras` (Keras model format). Received: filepath=my_cifar10_model
```

Các bước tiếp theo: Giải thích lỗi

```
%load_ext tensorboard
%tensorboard --logdir=./my_cifar10_logs
```

```
model.fit(X_train, y_train, epochs=100,
          validation_data=(X_valid, y_valid),
          callbacks=callbacks)
```

```
model.evaluate(X_valid, y_valid)
```

Mô hình có **độ lỗi xác thực (validation loss)** thấp nhất đạt độ chính xác khoảng **46.8%** trên tập xác thực. Phải mất 29 **epoch** để đạt được độ lỗi xác thực thấp nhất, với khoảng 10 giây mỗi epoch trên máy tính xách tay của tôi (không có **GPU**). Hãy xem liệu chúng ta có thể cải thiện mô hình bằng cách sử dụng **Chuẩn hóa Hàng loạt (Batch Normalization)**.

c.

*Bài tập: Bây giờ hãy thử thêm **Chuẩn hóa Hàng loạt (Batch Normalization)** và so sánh các **đường cong học tập (learning curves)**: Nó có **hội tụ nhanh hơn** trước không? Nó có tạo ra một **mô hình tốt hơn** không? Nó ảnh hưởng đến **tốc độ huấn luyện** như thế nào?*

Đoạn mã dưới đây rất giống với đoạn mã ở trên, với một vài thay đổi:

- Tôi đã thêm một lớp **BN (Batch Normalization)** sau mỗi lớp **Dense** (trước hàm kích hoạt), ngoại trừ lớp đầu ra.
- Tôi đã thay đổi **tốc độ học** thành $5e-4$. Tôi đã thử nghiệm với các giá trị $1e-5, 3e-5, 5e-5, 1e-4, 3e-4, 5e-4, 1e-3$ và $3e-3$, và tôi đã chọn giá trị có **hiệu suất xác thực** tốt nhất sau 20 **epoch**.
- Tôi đã đổi tên các thư mục chạy thành `run_bn_*` và tên tệp mô hình thành `my_cifar10_bn_model`.

```
tf.random.set_seed(42)

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[32, 32, 3]))
for _ in range(20):
    model.add(tf.keras.layers.Dense(100, kernel_initializer="he_normal"))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation("swish"))

model.add(tf.keras.layers.Dense(10, activation="softmax"))

optimizer = tf.keras.optimizers.Nadam(learning_rate=5e-4)
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=optimizer,
              metrics=["accuracy"])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=20,
                                                     restore_best_weights=True)
model_checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_cifar10_bn_model",
                                                         save_best_only=True)
run_index = 1 # increment every time you train the model
run_logdir = Path() / "my_cifar10_logs" / f"run_bn_{run_index:03d}"
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir)
callbacks = [early_stopping_cb, model_checkpoint_cb, tensorboard_cb]

model.fit(X_train, y_train, epochs=100,
          validation_data=(X_valid, y_valid),
          callbacks=callbacks)

model.evaluate(X_valid, y_valid)
```

- *Mô hình có **hội tụ nhanh hơn** trước không?* **Nhanh hơn nhiều!** Mô hình trước đó mất 29 **epoch** để đạt được độ lỗi xác thực thấp nhất, trong khi mô hình mới đạt được cùng mức độ lỗi đó chỉ trong **12 epoch** và tiếp tục có tiến triển cho đến epoch thứ 17. Các lớp **BN (Batch Normalization)** đã **ổn định quá trình huấn luyện** và cho phép chúng ta sử dụng **tốc độ học lớn hơn** nhiều, do đó sự hội tụ nhanh hơn.
- *BN có tạo ra **mô hình tốt hơn** không?* **Có!** Mô hình cuối cùng cũng tốt hơn nhiều, với độ chính xác xác thực là **50.7%** thay vì 46.7%. Nó vẫn chưa phải là một mô hình rất tốt, nhưng ít nhất thì nó tốt hơn nhiều so với trước đây (**Mạng Nơ-ron Tích chập - Convolutional Neural Network** sẽ làm tốt hơn nhiều, nhưng đó là một chủ đề khác, xem chương 14).

- *BN ảnh hưởng đến **tốc độ huấn luyện** như thế nào?* Mặc dù mô hình hội tụ nhanh hơn nhiều, mỗi **epoch** mất khoảng **15 giây** thay vì 10 giây, do các phép tính bổ sung cần thiết của các lớp BN. Nhưng nhìn chung **thời gian huấn luyện (wall time)** để đạt được mô hình tốt nhất đã được **rút ngắn khoảng 10%**.

d.

*Bài tập: Hãy thử thay thế **Chuẩn hóa Hàng loạt (Batch Normalization)** bằng **SELU**, và thực hiện các điều chỉnh cần thiết để đảm bảo mạng **tự chuẩn hóa (self-normalizes)** (tức là **chuẩn hóa** các **đặc trưng đầu vào (input features)**, sử dụng **khởi tạo bình thường LeCun**, đảm bảo **Mạng Nơ-ron Sâu (DNN)** chỉ chứa một chuỗi các lớp **dense**, v.v.).*

```
tf.random.set_seed(42)

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[32, 32, 3]))
for _ in range(20):
    model.add(tf.keras.layers.Dense(100,
                                    kernel_initializer="lecun_normal",
                                    activation="selu"))

model.add(tf.keras.layers.Dense(10, activation="softmax"))

optimizer = tf.keras.optimizers.Nadam(learning_rate=7e-4)
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=optimizer,
              metrics=["accuracy"])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    patience=20, restore_best_weights=True)
model_checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
    "my_cifar10_selu_model", save_best_only=True)
run_index = 1 # increment every time you train the model
run_logdir = Path() / "my_cifar10_logs" / f"run_selu_{run_index:03d}"
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir)
callbacks = [early_stopping_cb, model_checkpoint_cb, tensorboard_cb]

X_means = X_train.mean(axis=0)
X_stds = X_train.std(axis=0)
X_train_scaled = (X_train - X_means) / X_stds
X_valid_scaled = (X_valid - X_means) / X_stds
X_test_scaled = (X_test - X_means) / X_stds

model.fit(X_train_scaled, y_train, epochs=100,
          validation_data=(X_valid_scaled, y_valid),
          callbacks=callbacks)

model.evaluate(X_valid_scaled, y_valid)
```

Mô hình này đạt đến **độ lỗi xác thực** của mô hình đầu tiên chỉ trong **8 epoch**. Sau 14 epoch, nó đạt đến **độ lỗi xác thực thấp nhất**, với độ chính xác khoảng **50.3%**, tốt hơn so với mô hình gốc (46.7%), nhưng không bằng mô hình sử dụng **chuẩn hóa hàng loạt (batch normalization)** (50.7%). Mỗi epoch chỉ mất **9 giây**. Vì vậy, đây là mô hình **huấn luyện nhanh nhất** cho đến nay.

e.

*Bài tập: Hãy thử **điều chuẩn hóa** mô hình bằng **alpha dropout**. Sau đó, **mà không cần huấn luyện lại** mô hình, hãy xem liệu bạn có thể đạt được độ chính xác tốt hơn bằng cách sử dụng **MC Dropout**.*

```
tf.random.set_seed(42)

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[32, 32, 3]))
for _ in range(20):
    model.add(tf.keras.layers.Dense(100,
                                    kernel_initializer="lecun_normal",
                                    activation="selu"))

model.add(tf.keras.layers.AlphaDropout(rate=0.1))
model.add(tf.keras.layers.Dense(10, activation="softmax"))

optimizer = tf.keras.optimizers.Nadam(learning_rate=5e-4)
```

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=optimizer,
              metrics=["accuracy"])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    patience=20, restore_best_weights=True)
model_checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
    "my_cifar10_alpha_dropout_model", save_best_only=True)
run_index = 1 # increment every time you train the model
run_logdir = Path() / "my_cifar10_logs" / f"run_alpha_dropout_{run_index:03d}"
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir)
callbacks = [early_stopping_cb, model_checkpoint_cb, tensorboard_cb]

X_means = X_train.mean(axis=0)
X_stds = X_train.std(axis=0)
X_train_scaled = (X_train - X_means) / X_stds
X_valid_scaled = (X_valid - X_means) / X_stds
X_test_scaled = (X_test - X_means) / X_stds

model.fit(X_train_scaled, y_train, epochs=100,
          validation_data=(X_valid_scaled, y_valid),
          callbacks=callbacks)

model.evaluate(X_valid_scaled, y_valid)
```

Mô hình đạt độ chính xác **48.1%** trên **tập xác thực (validation set)**. Kết quả này **tệ hơn** so với khi không có **dropout** (50.3%). Với một quá trình tìm kiếm **siêu tham số (hyperparameter search)** sâu rộng, có thể đạt được kết quả tốt hơn (tôi đã thử các tỷ lệ dropout 5%, 10%, 20% và 40%, cùng với các tốc độ học $1e-4, 3e-4, 5e-4$ và $1e-3$), nhưng có lẽ trong trường hợp này cũng không thể tốt hơn nhiều.

Bây giờ chúng ta hãy sử dụng **MC Dropout**. Chúng ta sẽ cần lớp `MCAlphaDropout` mà chúng ta đã sử dụng trước đó, vì vậy hãy sao chép nó vào đây để tiện lợi:

```
class MCAlphaDropout(tf.keras.layers.AlphaDropout):
    def call(self, inputs):
        return super().call(inputs, training=True)
```

Bây giờ chúng ta hãy tạo một mô hình mới, **giống hệt** mô hình chúng ta vừa huấn luyện (với cùng **trọng số**), nhưng với các lớp **dropout** là `MCAlphaDropout` thay vì các lớp `AlphaDropout`:

```
mc_model = tf.keras.Sequential([
    (
        MCAlphaDropout(layer.rate)
        if isinstance(layer, tf.keras.layers.AlphaDropout)
        else layer
    )
    for layer in model.layers
])
```

Sau đó, chúng ta hãy thêm một vài **hàm tiện ích (utility functions)**. Hàm đầu tiên sẽ chạy mô hình nhiều lần (mặc định là 10) và nó sẽ trả về **xác suất lớp được dự đoán trung bình (mean predicted class probabilities)**. Hàm thứ hai sẽ sử dụng các xác suất trung bình này để dự đoán **lớp có khả năng nhất** cho mỗi cá thể:

```
def mc_dropout_predict_probas(mc_model, X, n_samples=10):
    Y_probas = [mc_model.predict(X) for sample in range(n_samples)]
    return np.mean(Y_probas, axis=0)

def mc_dropout_predict_classes(mc_model, X, n_samples=10):
    Y_probas = mc_dropout_predict_probas(mc_model, X, n_samples)
    return Y_probas.argmax(axis=1)
```

Bây giờ chúng ta hãy đưa ra **dự đoán** cho tất cả các cá thể trong **tập xác thực (validation set)**, và tính toán **độ chính xác**: